

ZENITHIUM

TEKNİK DÖKÜMAN

Tamamı cihaz üzerinde çalışan bir toparlanma, antrenman ve sağlık zekâsı uygulaması. iOS 18, Swift 6, sıfır sunucu.

DEPO KÜNYESİ — 21 AĞUSTOS 2026 · İLK YAYIN

DEPO	projectcagla/zenithra
SÜRÜM	0.1 (build 1) — MARKETING_VERSION , üç Info.plist ve motor sürümü tek kaynaktan
DAL	claude/apexengine-ios-build-4zagbl · HEAD 272da6a
KAYNAK	295 Swift dosyası · 55.762 satır
HEDEFLER	4 — iOS · watchOS · Widget uzantısı · Test
MOTORLAR	29 saf hesap motoru · 6.208 satır
TESTLER	588 test · 48 dosya · 66 paket
VERİ MODELİ	12 SwiftData modeli · şema V1 → V3
BAĞIMLILIK	0 üçüncü parti paket · 0 ağ çağrısı · 0 hesap
COMMIT	65 commit · 33 faz + v4 + sertleştirme + yayın taraması

BELGE SÜRÜMÜ ARTIK UYGULAMA SÜRÜMÜNÜ İZLİYOR — ÖNCEKİ BASKI YOL HARİTASI NUMARASINI (4.2) TAŞIYORDU, BU BASKI YAYINLANAN SÜRÜMÜ TAŞIYOR
BU BELGE DEPODAKİ KAYNAK KODDAN ÜRETİLMİŞTİR · ÖLÇÜMLER WC, GREP VE GIT ÇIKTILARIDIR

BÖLÜMLER

01	Ürün	4
	Zenithium ne yapar, kimin için, hangi kuralları asla çiğnemez	
02	Mimari	7
	Katmanlar, bağımlılık yönü, Swift 6 eşzamanlılık modeli, dört hedef	
03	Veri katmanı	11
	SwiftData modelleri, şema göçü, HealthKit okuma hattı, önbellek	
04	Motorlar	14
	29 saf hesap motoru — girdi, çıktı, dayandığı bilim, kritik sabitler	
05	Arayüz ve tasarım sistemi	23
	Gezinme, ekran envanteri, renk/tipografi altyapısı, erişilebilirlik	
06	Sistem entegrasyonu	26
	Apple Watch, widget'lar, Live Activity, App Intents, arka plan	
07	Zekâ katmanı	28
	Cihaz içi dil modeli, güvenlik filtresi, laboratuvar OCR hattı	
08	Veri taşınabilirliği	31
	Arşiv dışı/içe aktarım, hekim raporu, GPX, belge kasası	
09	Teknoloji yığını	33
	Kullanılan 20 birinci parti çatı, nerede ve neden	
10	Kalite ve doğrulama	35
	588 test, test stratejisi, varsayım kayıt defteri, yayın öncesi tarama	
11	Yol haritası v4	39
	Performans, tasarım, özellikler ve altı adımlık sertleştirme turu	
12	Geliştirme süreci	43
	Depo düzeni, üreteç betikleri, yayın öncesi durum, derleme kontrolleri	

A	Ek A — Faz listesi	47
	Faz 0'dan Faz 32'ye ne inşa edildi	
B	Ek B — Varsayım kayıt defteri	48
	106 numaralı varsayım, kategorileri ve riskli olanlar	

BU BELGE HAKKINDA

Bu döküman Zenithium'un bugünkü kaynak kodundan üretildi. İçindeki her sayı — dosya sayısı, satır sayısı, test sayısı, motor uzunlukları — depoda çalıştırılan bir komutun çıktısıdır, hafızadan yazılmış bir tahmin değil. Nerede bir şey ölçülmediyse veya doğrulanmadıysa, bunu açıkça söylüyorum.

DOĞRULAMA SINIRI

Bu kod hiç derlenmedi. Geliştirme, Swift araç zinciri bulunmayan bir Linux konteynerinde yapıldı. Motorların matematiği bağımsız olarak Python'da yeniden hesaplanıp doğrulandı, hedefler arası kaynak kapanışı statik taramayla kontrol edildi, ama *hiçbir dosya Swift derleyicisinden geçmedi ve hiçbir test koşturulmadı*. İlk `xcodebuild` çalıştırmasında sözdizimi ve tip hataları çıkması beklenmelidir. Belgede "588 test" derken kastedilen, yazılmış ve `@Test` ile işaretlenmiş test sayısıdır — geçmiş test sayısı değil.

Zenithium, bir kullanıcının kendi vücudundan gelen sinyalleri okuyup ona bugün ne yapması gerektiğini söyleyen, tamamen telefonun içinde çalışan bir uygulamadır. Whoop'un cevapladığı soruyu cevaplar — *bugün ne kadar yüklenebilirim?* — ama tek bir sporcu tipini değil dördünü birden ciddiye alır.

1.1 Problem

Toparlanma takip eden uygulamalar iki uçtan birinde duruyor. Bir uçta abonelikli donanım ekosistemleri var: kapalı algoritma, buluta giden veri, aylık ücret ve tek bir sporcu profili — genellikle "genel dayanıklılık". Diğer uçta ham veri gösteren panolar var: sayıyı veriyor, ne anlama geldiğini söylemiyor.

Zenithium ikisinin arasını hedefliyor: **ham veriyi karar cümlesine çeviren, ama bunu nasıl yaptığını açıklayan ve veriyi cihazdan hiç çıkarmayan** bir uygulama. Kullanıcı sabah uygulamayı açtığında bir yüzde değil, bir cümle görüyor — ve o cümlenin altında, isterse, o cümleyi üreten sayılar duruyor.

1.2 Dört mercecek

Uygulamanın merkezî tasarım kararı *mercek* kavramıdır. Toparlanma ve uyku hesabı herkes için aynı çalışır; ama o toparlanma skorunun *ne anlama geldiği* kullanıcının ne yaptığına bağlıdır. Bir maratoncunun %62 toparlanması ile bir powerlifter'ın %62'si aynı öneriyi doğurmaz. Mercek, ortak çekirdeğin üzerine oturan yorum katmanıdır.

DÖRT MERCEK VE HER BİRİNİN SORDUĞU SORU

MERCEK	KULLANICI	MERCEĞİN CEVAPLADIĞI SORU
Dayanıklılık	Koşucu, bisikletçi, triatlet	Kritik hızım nerede, bu tempo sürdürülebilir mi, yarış planım gerçekçi mi?
Hibrit	Hyrox, CrossFit, engelli koşu	Aerobik ve kuvvet yükünü aynı hafta içinde nasıl dengeliyorum?
Kuvvet	Powerlifting, bodybuilding	Hangi kas grubum hâlâ yorgun, tonaj eğilimim nereye gidiyor, 1RM'im?
Sağlık	Spor yapmayan, sadece izleyen	Vücudum normalinden sapıyor mu, uykum ve ritmim ne durumda?

Mercek seçimi kalıcı bir kimlik değil, bir ayardır: kullanıcı istediği zaman değiştirir, hatta sağlık merceği hiç antrenman yapmayan biri için varsayılan olarak çalışır. Sekme çubuğunun ikinci sekmesi

merceğe göre değişir — dayanıklılık ve hibrit merceğinde "Yük", kuvvette "Kaslar", sağlıkta "Sağlık".

1.3 Günlük döngü

Uygulama üç ana anı etrafında kurgulanmıştır.

AN	NE OLUYOR	KULLANICININ GÖRDÜĞÜ
Sabah	Gece verisi işlenir, taban çizgileri güncellenir, reçete üretilir	Toparlanma bandı, tek cümlelik reçete, o cümlenin gerekçesi
Gün	Antrenman kaydedilir, canlı seans saatten yayınlanır, zorlanma birikir	Canlı seans kartı (kilit ekranı + Dynamic Island), zorlanma halkası
Akşam	Günlük soruları, ağrı girişi, ertesi gün için plan	Günlük formu, korelasyon bulguları, yarınki oturumun şekli

1.4 Çiğnenmeyen iki kural

Spesifikasyonun iki maddesi ürün boyunca bağlayıcıdır ve kodda test edilir.

ŞARTNAME §1

Uygulamanın hiçbir yerinde **kalori hedefi, kilo hedefi veya kısıtlama teşviki** bulunmaz. Beslenme takibi yoktur. "Şu kadar kalori açığı ver" türü hiçbir cümle üretilemez. Bu, yeme bozukluğu riskini ürünün dışında tutmak için verilmiş bilinçli bir karardır ve bir özellik eksikliği değildir.

ŞARTNAME §12

Zenithium **bir tıbbi cihaz değildir**. Teşhis koymaz, koyduğunu ima etmez, ve hiçbir koşulda kullanıcıya bir belirtiyi görmezden gelmesini söylemez. Cihaz içi dil modelinin ürettiği her cümle `SafetyFilter` 'dan geçer; teşhis diline benzeyen çıktı reddedilir ve yerine deterministik şablon metin konur.

1.5 Gizlilik duruşu

Zenithium'un mimarisi bir gizlilik politikasından değil, bir kısıttan doğdu: **uygulamanın ağ katmanı yok**. Bu bir tercih beyanı değil, denetlenebilir bir olgudur — depoda `URLSession`, `URLRequest` veya herhangi bir ağ çağrısı geçmez.

YOK OLAN ŞEY	SONUCU
Sunucu	Hesap yok, giriş ekranı yok, parola yok, sunucu ihlali yok

YOK OLAN ŐEY	SONUCU
Ađ çağrısı	Uçak modunda uygulamanın tamamı çalışır
Analitik	Crash raporu dahil hiçbir telemetri gönderilmez
Üçüncü parti paket	Tedarik zinciri yüzeyi sıfır; SPM bağımlılık listesi boş
Reklam kimliği	ATT diyalogu hiç gösterilmez

Veri kullanıcının cihazında SwiftData deposunda durur; dışa aktarım kullanıcının kendi başlattığı, kendi seçtiğı bir dosyaya yazan bir işlemdir (bkz. Bölüm 8). Cihaz içi dil modeli `FoundationModels` ile çalışır, yani üretilen metin de cihazdan çıkmaz.

Zenithium tek yönlü bir bağımlılık zinciri üzerine kuruludur. En altta hiçbir şeye bağımlı olmayan saf değer tipleri, en üstte SwiftUI görünümleri vardır. Bu zincirin yönü bir stil tercihi değil, Apple Watch ve widget hedeflerinin derlenebilmesinin ön koşuludur.

2.1 Katmanlar

```
// Bağımlılık yönü – yukarıdan aşağıya, asla tersi.

Views           SwiftUI görünümleri, Canvas çizimleri, tasarım sistemi
↓
ViewModels      @MainActor @Observable durum sahipleri
↓
Orchestration   Günlük yeniden hesaplama, arka plan görevleri, göç
Persistence     SwiftData deposu, repository protokolleri, önbellek
Health          HealthKit okuma hattı, canlı seans oturumu
Intelligence     Cihaz içi dil modeli + güvenlik filtresi
↓
Engines         29 saf hesap motoru – static func, durum yok
↓
Domain          Saf değer tipleri. Yalnızca Foundation. Hiçbir şeye bağımlı değil.
```

Kural şudur: **bir katman yalnızca kendisinden aşağıdakileri import edebilir.** Motorlar Domain'i görür; Domain motorları görmez. Görünümler ViewModel'i görür; ViewModel görünümü görmez.

NEDEN BU KURAL İHLAL EDİLEMEZ

Apple Watch uygulaması ve widget uzantısı, iOS uygulamasının kaynak dosyalarının yalnızca bir *alt kümesini* derler. Watch hedefi `Domain` 'den birkaç dosya alır ama `Engines` 'i almaz. Eğer bir `Domain` dosyası bir motora atıfta bulunursa, iOS hedefi sorunsuz derlenir ama **watch hedefi kırılır** — ve bu, iOS'ta hiçbir belirti vermez.

Bu tam olarak iki kez oldu: `SleepDebtIO.swift` `SleepDebtEngine` 'e, `HeatAcclimation.swift` `HeatAcclimationEngine` 'e sızdı. İkisi de her yığın için çalıştırılan statik kaynak-kapanış taramasıyla yakalandı. Çözüm eşiği değer tipine taşımaktı:

```
SleepDebtLedger.minimumNights,
HeatAcclimationState.establishedThreshold — motor artık kendi
sabitini değer tipinden okuyor.
```

2.2 Eşzamanlılık modeli

Proje **Swift 6 dil kipinde, sıkı eşzamanlılık denetimi "complete"** ayarıyla yazıldı. Bu, veri yarışlarını çalışma zamanında değil derleme zamanında yakalar — ve bedeli, her tipin izolasyon alanının açıkça belirtilmesidir.

EŞZAMANLILIK ENVANTERİ — ÖLÇÜM

YAPI	SAYI	ROLÜ
actor	10	Paylaşılan değişken durumun tek sahibi — depo, önbellek, HealthKit köprüsü
@ModelActor	2	SwiftData <code>ModelContext</code> 'ini kendi izolasyonunda tutan depo aktörleri
@Observable	22	ViewModel'ler — hepsi <code>@MainActor</code>
@MainActor	37	Arayüze dokunan her tip
Sendable uyumu	273	İzolasyon sınırlarını geçen her değer tipi

YASAK LİSTESİ VE DENETİMİ

Şartname §2.4 bir dizi yapıyı yasaklar. Bugünkü durum, depoda çalıştırılan bir aramanın sonucu:

YASAKLI YAPI	BULUNAN	YERİNE KULLANILAN
ObservableObject / @Published	0	<code>@Observable</code> makrosu
Combine	0	<code>AsyncSequence</code> , <code>AsyncStream</code>
DispatchQueue	0 1 YORUM	actor izolasyonu, <code>Task</code>
NSLock	0	actor izolasyonu
print()	0	<code>OSLog</code> / <code>OSSignposter</code>
Zorlamalı açma <code>!</code> , <code>try!</code> , <code>as!</code>	0	<code>guard let</code> , <code>throws</code> , <code>as?</code>
Değişken global durumlu singleton	0	Bağımlılık enjeksiyonu, protokol soyutlaması
Storyboard	0	Tamamı SwiftUI
UIViewRepresentable	0	Yerel SwiftUI; iki <code>UIKit</code> import'u var — <code>ReportDocumentRenderer</code> 'da PDF çizimi, <code>ProtectedDataGuard</code> 'da veri koruma durumu. İkisinin de karşılığı yok ve gerekçeleri dosyalarında yazılı

Yasak listesindeki tek "1 yorum" işareti, `BackgroundRefreshScheduler.swift` içinde neden `DispatchQueue` kullanılmadığını açıklayan bir yorum satırıdır — çağrı değildir.

SINIRLI PARALELLİK DESENİ

Birden çok bağımsız okuma gerektiğinde kalıp şudur: sıralı `await` zinciri değil, **eşzamanlılığı sınırlanmış bir** `withTaskGroup`. Sınır

olmadan 18 HealthKit sorgusu aynı anda açılır ve HealthKit'in kendi kuyruğu boğulur.

```
private enum VitalReadOutcome: Sendable {
    case samples([VitalSample])
    case blocked(ZenithiumError) // izin reddedildi – kullanıcıya söylenir
    case unreadable // veri yok – sessizce atlanır
}

private static let maximumConcurrentVitalReads = 4
```

Aynı desen `HealthKitService` içindeki iki döngüde de uygulandı. Hata, atılan bir `Error` yerine bir `Sendable` enum olarak taşınır; çünkü "izin yok" ile "veri yok" kullanıcı için farklı anlamlara gelir ve bir `try` zinciri ikisini ayırt edemez.

2.3 Dört hedef

HEDEFLER VE KAYNAK PAYLAŞIMI

HEDEF	PLATFORM	DOSYA	KAPSAM
Zenithium	iOS 18	246	Ana uygulama — bütün katmanlar
ZenithiumWatch	watchOS 11	7	Canlı seans, <code>HKWorkoutSession</code> , telefona yayın
ZenithiumWidgets	iOS 18	9	7 widget + Live Activity + Control Widget
ZenithiumTests	iOS 18	48	588 test · 66 paket

Watch ve widget hedefleri ana uygulamanın `Domain` dosyalarından seçtiklerini *doğrudan derler* — paylaşılan bir çerçeve (framework) hedefi yoktur. Bu, ek bir modül sınırı ve dinamik bağlama maliyeti getirmemek için verilmiş bir karardır; bedeli, §2.1'deki bağımlılık kuralının elle korunmasıdır.

Örnek: watch uygulaması zorlanma (strain) göstergesi çizer ama `RecoveryEngine`'i derlemez. Bu yüzden TRIMP↔zorlanma dönüşümü `RecoveryEngine`'den çıkarılıp bağımsız bir `Domain/StrainScale.swift` dosyasına taşındı; her iki hedef de aynı ölçeği aynı sabitlerle kullanır.

2.4 Klasör envanteri

ÖLÇÜM — wc -L, 21 AĞUSTOS 2026 · SÜRÜM 0.1

KLASÖR	DOSYA	SATIR	İÇERİĞİ
Views	55	11.503	19 ekran ailesi + tasarım sistemi
Domain	67	10.807	Saf değer tipleri, 14 alt alan
Engines	29	6.208	Hesap motorları
ViewModels	21	4.030	<code>@MainActor</code> <code>@Observable</code> durum
Persistence	11	2.833	SwiftData deposu, önbellek, arşiv
Health	6	2.419	HealthKit okuma ve canlı oturum
Orchestration	6	1.557	Günlük hesap, arka plan, göç

KLASÖR	DOSYA	SATIR	İÇERİĞİ
Models	16	1.276	12 SwiftData modeli + 3 şema sürümü
Reports	3	472	Hekim raporu üretimi
Intelligence	3	338	Dil modeli + güvenlik filtresi
Labs	2	307	PDF/OCR ayrıştırma
Connectivity	1	203	WatchConnectivity köprüsü
Endurance	1	189	GPX okuyucu
Support	3	182	Biçimlendirme, signpost, hata tipleri
Intents	1	155	App Intents / Siri
App	3	473	Giriş noktası, kök gezinme
Live	1	61	ActivityKit öznelikleri
ZenithiumWidgets	9	949	Widget uzantısı
ZenithiumWatch	7	874	watchOS uygulaması
ZenithiumTests	38	8.346	Test paketleri

VERİ KATMANI

Verinin tek bir yönü vardır: HealthKit'ten okunur, motorlarda işlenir, SwiftData'ya günlük özet olarak yazılır. Uygulama HealthKit'e asla yazmaz ve ham örnekleri kalıcı olarak saklamaz — yalnızca günlük türetilmiş değerleri.

3.1 SwiftData modelleri

12 KALICI MODEL

MODEL	NE SAKLAR
BiometricDayRecord	Bir günün tüm türetilmiş değerleri: HRV, RHR, bilek sıcaklığı, solunum hızı, uyku skoru, toparlanma, zorlanma, motor sürümü. Uygulamanın merkezî tablosu.
BaselineState	Metrik başına EWMA taban çizgisi, standart sapması ve örnek sayısı. Her sabah güncellenir.
UserProfile	Yaş, cinsiyet, boy, mercek seçimi, hedefler, görünüm tercihi. Tek kayıt.
JournalDayLog	Günlük öznel girişler: enerji, ruh hali, alkol, kafein, stres, notlar. Korelasyon motorunun girdisi.
StrengthSessionLog	Kuvvet seansı: hareket, set, tekrar, ağırlık, RPE. Tonaj ve 1RM tahmininin kaynağı.
HybridSessionLog	Hyrox/CrossFit seansı: bölümler, süreler, iş türü.
MuscleFatigueSnapshot	Kas grubu başına gün sonu yorgunluk değeri; kas haritasının beslediği tablo.
PainEntryLog	Bölge, şiddet, tip, tarih. Ağrı haritası ve erken uyarı için.
BloodMarker	Laboratuvar sonucu: biyobelirteç, değer, birim, tarih, referans aralığı, kaynak belge.
HealthDocumentLog	Belge kasası girdisi. PDF <code>@Attribute(.externalStorage)</code> ile dosya sisteminde tutulur, çıkarılan metin aranabilir.
GoalEventLog	Yarış/hedef olayı: tarih, mesafe, tip. Planlama ve tepe noktası hesabı.
SupplementCourseLog	Takviye kürü: başlangıç, bitiş, doz. Öncesi/sonrası karşılaştırma için.

Tarih anahtarlı modellerde `@Attribute(.unique)` kullanılır, böylece aynı gün iki kez hesaplanırsa kayıt çoğalmaz, güncellenir.

3.2 Şema sürümleme

Şema `VersionedSchema` ile üç sürüme ayrıldı ve `ZenithiumMigrationPlan` iki hafif göç aşaması tanımlar:

GEÇİŞ	TİP	DEĞİŞİKLİK
V1 → V2	lightweight	<code>SupplementCourseLog</code> eklendi; mevcut hiçbir alan değişmedi
V2 → V3	lightweight	<code>UserProfile.appearanceRawValue</code> eklendi, varsayılanlı

Her ikisi de yalnızca *ekleme* olduğu için `SwiftData` dönüşüm kodu olmadan göç edebiliyor. Dosyanın başında sonraki katkı sağlayıcı için dört adımlık bir yordam yazılı: V4 enum'unu kopyala, `schemas` 'a ekle, aşamayı ekle, ve hesaplanan değerler değiştiyse `engineVersion` 'ı yükselt.

MOTOR SÜRÜMÜ MEKANİZMASI

`BiometricDayRecord` hangi motor sürümüyle hesaplandığını saklar. Bir motorun matematiği değişirse `EngineConstants.Orchestration.engineVersion` artırılır; eski sürümle yazılmış kayıtlar bayat sayılır ve arka planda yeniden hesaplanır. Böylece kullanıcı bir güncellemeneden sonra tutarsız bir geçmiş görmez.

3.3 HealthKit okuma hattı

Uygulama sekiz HealthKit veri türüne *yalnızca okuma* izni ister:

`heartRate`, `heartRateVariability`, `restingHeartRate`, `respiratoryRate`, `oxygenSaturation`, `wristTemperature`, `sleepAnalysis`, `workout`. Sağlık merceği ayrıca 18 hayati işaret metriğini okur (yürüme hızı, adım uzunluğu, merdiven çıkış hızı, gün ışığı süresi, kulaklık ses maruziyeti dâhil).

SORGU TÜRÜ	KULLANIM
HKStatisticsCollectionQuery	Günlük özetler — gün başına ortalama/toplam. Taban çizgisi beslemesi.
HKAnchoredObjectQuery	Artımlı senkronizasyon — son çapadan beri eklenen/silinen örnekler.
HKWorkoutSession + HKLiveWorkoutBuilder	Watch'ta canlı seans; kalp hızı ve enerji akışı.
HKMetadataKeyWeather...	Antrenman meta verisinden sıcaklık ve nem — ısı adaptasyonu motorunun girdisi.

GERİYE DOLDURMA

İlk kurulumda 60 günlük geçmiş doldurulur, ama tek seferde değil: her geçişte en fazla 7 gün işlenir ve aradaki adımlarda ana iş parçacığı serbest bırakılır. Böylece kurulum ekranı donmaz ve HealthKit hız sınırına takılmaz.

3.4 Depo soyutlaması ve önbellek

Motorlar `SwiftData`'yı hiç görmez. Aralarındaki sözleşme bir protokoldür — `BiometricDayRepository` — ve gerçek uygulaması bir `@ModelActor` 'dır. Testlerde aynı protokolün bellek-içi sahtesi kullanılır; bu, 588 testin hiçbirinin bir veritabanına ihtiyaç duymamasının sebebidir.

Bu soyutlamanın üzerine yol haritası v4'te bir önbellek katmanı eklendi. Sorun ölçülmüştü: bir sabah yeniden hesaplamasında sekiz farklı motor, *aynı* 60 günlük pencereyi ayrı ayrı istiyordu.

```
private static let minimumHistoryDays: Double = 150
private static let forwardSlackDays: Double = 1

func dayRecords(from start: Date, through end: Date) async throws ->
[BiometricDaySnapshot] {
    requestCount += 1
    if let window, window.start <= start, window.end >= end {
        return slice(window.days, from: start, through: end) // disk yok
    }
    // Pencereyi genişlet: hem geriye 150 güne, hem ileriye 1 gün paya.
    let widenedEnd = max(end, window?.end ?? end).addingTimeInterval(forwardSlackDays
* 86_400)
    let floor = widenedEnd.addingTimeInterval(-minimumHistoryDays * 86_400)
    let widenedStart = min(start, window?.start ?? start, floor)
    let days = try await upstream.dayRecords(from: widenedStart, through: widenedEnd)
    upstreamReadCount += 1
    window = (widenedStart, widenedEnd, days)
    return slice(days, from: start, through: end)
}
```

`DayRecordCache` bir `actor`'dır ve aynı protokole uyar; yani çağırın taraf için görünmezdir. Genişletme stratejisi kasıtlıdır: 150 günlük taban, en uzun geçmiş isteyen motorun (ACWR + mevsimsel karşılaştırma) ihtiyacını tek okumada karşılar. `requestCount` ve `upstreamReadCount` sayaçları yalnızca test içindir — `DayRecordCacheTests` sekiz isteğin bir okumaya indiğini doğrular.

MOTORLAR

Uygulamanın hesap yapan her satırı 29 motorda toplanmıştır. Hepsi `enum` içinde `static func` 'tir: durum tutmazlar, yan etkileri yoktur, yalnızca `Foundation` 'a bağlıdırlar ve aynı girdiye her zaman aynı çıktıyı verirler. Bu, 569 testin varlık sebebidir — bir motoru test etmek için ne cihaz ne veritabanı ne de zaman gerekir.

29	6.208	342	0
MOTOR	SATIR	SABİT SATIRI	YAN ETKİ

SABİTLERİN TEK YERİ

`EngineConstants.swift` (342 satır) her motorun eşiklerini tek bir dosyada toplar; her sabitin yanında *neden o değer olduğunu* açıklayan bir yorum bulunur. Bir eşiği değiştirmek, o eşiğin gerekçesini de değiştirmeyi gerektirir — sayı ile gerekçe aynı yerde durduğu için sessizce ayrılamazlar.

4.1 Temel katman

BaselineEngine

226 satır

Her metriğin 60 günlük üstel ağırlıklı hareketli ortalaması (EWMA) ve standart sapması. Uygulamadaki "normalin" tanımı buradan gelir; toparlanma, hayati işaretler ve erken uyarı hepsi bu tabana bakar.

Beş koruma kuralı çağrı yerlerinde değil burada uygulanır: **(1)** bugün, kendisinin puanlandığı tabanı kirletmez — `scoringBaseline(from:)` durumu dünden okur; **(2)** winsorization girişte, güncellemeden önce yapılır; **(3)** σ tabanları her σ 'ya uygulanır; **(4)** soğuk başlangıç 5 günün altında skoru bastırır, 14 günün altında öncüle doğru harmanlar; **(5)** eksik gün EWMA'yı ilerletmez — hiçbir yerde interpolasyon yoktur.

GİRDİ	Metrik geçmişi, mevcut taban durumu
ÇIKTI	<code>BaselineSnapshot</code> — μ , σ , örnek sayısı, kalibrasyon durumu
KURAL	Boş gün asla doldurulmaz; veri yoksa skor yoktur

MathSupport

174 satır

Paylaşılan istatistik: ortalama, standart sapma, yüzdelik, en küçük kareler eğimi, Welch standart hatası, Cohen'in d 'si, dairesel ortalama ve dairesel fark. Dairesel fonksiyonlar uyku orta noktası ve sirkadiyen hesapları için gereklidir — 23:50 ile 00:10 arasındaki fark 20 dakikadır, 23 saat 40 dakika değil.

MonotoneCubicInterpolator

144 satır

Fritsch–Carlson monotone kübik Hermite interpolasyonu. Grafik eğrilerinin ve sirkadiyen yayın veri noktaları arasında *aşmadan* geçmesini sağlar. Sıradan bir kübik spline iki uyku noktası arasında negatif uyanıklık üretebilir; monotone varyant bunu yapısal olarak imkânsız kılar.

EngineConstants

342 satır

Motor eşiklerinin tek kaynağı. Ayrıca `Orchestration.engineVersion` burada durur — matematik değiştiğinde artırılır ve eski kayıtların yeniden hesaplanmasını tetikler.

4.2 Toparlanma ve uyku

RecoveryEngine

235 satır

Uygulamanın en görünür sayısı. Dört fizyolojik sinyalin z-skorlarını ağırlıklandırıp lojistik bir eğriden 1–100 aralığına taşır.

```
Z_HRV = (HRV - μ) / σ
Z_RHR = -(RHR - μ) / σ
Z_Temp = -| ΔT / σ |
Z_Resp = -(BR - μ) / σ
SleepNorm = clamp((SleepScore - 70) / 15, -3, +3)
Z_total = Σ wi · Zi // ağırlıklar hayatta kalan terimlere yeniden
dağıtılır
Recovery = clamp(100 / (1 + e-1.2 · Z_total), 1, 100)
```

Sıcaklık terimi mutlak değerlidir ve daima negatiftir: hem yükselen hem düşen bilek sıcaklığı normalden sapmadır. Bir metrik eksikse ağırlığı silinmez, kalan terimlere *yeniden normalize* edilir — böylece üç metrikle ölçülen bir gün, dört metrikli bir günle aynı ölçekte kalır. Saat takılmadıysa skor üretilmez (`.unavailable(.noOvernightData)`); tahmin edilmez.

BANT Yeşil / sarı / kırmızı — reçete motorunun ilk girdisi

SINIR Hedef tavanı \$5.3 ile sınırlanır

SleepScoreEngine

274 satır

Dört bileşenli uyku skoru. Kritik nokta, *ihtiyacın sabit olmaması*: çünkü zorlanma ve birikmiş uyku borcu ihtiyacı yukarı, şekerleme kredisi aşağı çeker.

```
need_h = baselineNeed + 0.25 · (yesterdayStrain/21) + min(debt,
1.5) - napCredit

Duration = 100 · clamp(asleep_h / need_h, 0, 1) w 0.50
Efficiency = 100 · clamp((asleep/timeInBed - 0.75) / 0.20, 0, 1)
w 0.20
Restorative = 100 · clamp(((deep + REM) / asleep) / 0.42, 0, 1) w
0.20
Consistency = 100 · max(0, 1 - |midpoint - μ14d| / 90 dk) w 0.10
```

Tutarlılık bileşeni dairesel ortalama ile hesaplanır. Verimlilik %75'ten başlar çünkü altındaki değerler ölçüm hatasıyla ayırt edilemez.

SleepDebtEngine

V4 · C4

176 satır

14 günlük yuvarlanan uyku borcu defteri ve sosyal jetlag. Her gece **ihtiyaç - uyanan** kadar katkı verir.

İki tasarım kararı savunmayı hak ediyor. **Fazlalık sayılır ama tamamen değil** — on saatlik bir cumartesi, iki adet beş saatlik hafta içini geri almaz; fazlalık **surplusRepaymentFraction** oranında kredilendirilir. **Pencere yuvarlanır, sıfırlanmaz** — takvim haftası sınırında sıfırlanan bir defter, pazartesi sabahı borcu sihirli biçimde yok ederdi.

Sosyal jetlag Wittmann/Roenneberg tanımıyla hesaplanır: serbest günlerin uyku orta noktası ile iş günlerinin orta noktası arasındaki dairesel fark. Her iki tarafta da **SocialJetlag.minimumSideNights** kadar gece yoksa sonuç üretilmez.

CircadianEngine

177 satır

Gün içi uyanıklık yayı. Çapa noktaları uyku orta noktasına göre yerleştirilir ve dairesel 24 saatlik uzayda monotone kübik Hermite ile interpolate edilir. Çapa kümesi uyudurma öncesinde ± 24 saatte kopyalanır — bu, sarma dikişini görünür bir kırıklık yerine C^1 -sürekli yapar.

Örnek: uyku başlangıcı 23:30, süre 7,5 sa → orta nokta 03:15
→ zirve 07:45 · çukur 11:15 · ikincil zirve 14:45 · melatonin 18:15

4.3 Yük ve yorgunluk

StrainEngine

242 satır

Kardiyovasküler zorlanma. Banister TRIMP integrali kalp hızı serisinin üzerinden koşturulur, sonra doyuma giden bir üstel eğriyle 0–21 ölçeğine taşınır.

```
x(t) = clamp((HR(t) - RHR) / (HRmax - RHR), 0, 1)
Δt_i = clamp(t_i - t_{i-1}, 0, 60 sn) // > 120 sn boşluk hiç katkı vermez
TRIMP = Σ (Δt_i/60) · x_i · b · e^(c · x_i)
DailyStrain = 21 · (1 - e^(-0,0065 · TRIMP))
```

Boşluk kırpması önemlidir: saat çıkarılıp iki saat sonra takıldığında son örnek ile yeni örnek arasındaki fark iki saattir, ve kırpması olmasa bu iki saatlik yüksek yoğunluk olarak sayılırdı. **b** ve **c** katsayıları biyolojik cinsiyete göre değişir (Banister'ın orijinal kalibrasyonu).

Motor beş kalibrasyon çıkmasını $\pm 0,1$ içinde yeniden üretmek zorundadır ve bu testlerle sabitlenmiştir: TRIMP 65 → 7,2 · 135 → 12,3 · 200 → 15,3 · 365 → 19,0 · 500 → 20,2.

TrainingLoadEngine

249 satır

Tek bir günlük yük serisinden dört soruyu birden cevaplar.

ACWR	Akut:kronik oran, <i>üstel</i> olarak — iki hareketli ortalama çifti olarak değil. Yuvarlanan ortalama penceresinin yarısı kadar gecikir; pazartesi değişen bir hafta ancak perşembe sayıya yansır.
RAMPA	Haftadan haftaya değişim oranı
MONOTONLUK	Foster'ın monotonluk ve "strain" ölçüleri — haftanın ne kadar tekdüze olduğu
FORM	Banister iki bileşenli model: yavaş hareket eden fitness terimi, hızlı hareket eden yorgunluk terimi, form ikisinin farkı

Motor hangi sporun yapıldığını bilmez; mercekler sayılar hakkında ne söyleneceğine karar verir. **§12 notu dosyada yazılıdır:** iş yükü literatürü sakatlıkla *ilişki* bildirir, nedensellik değil — bu motor daha güçlü iddiaya yetkili değildir ve hiçbir bant bunu ima etmez.

FatigueEngine

150 satır

16 kas grubu için üstel yorgunluk sönümü. Her seansın kas başına etkisi bağımsız olarak üst üste biner — aynı gün iki antrenman toplanır, birbirinin yerini almaz.

```
sleepModifier = clamp(1,35 - 0,006 * SleepScore, 0,75, 1,35)
t½_m = 24 sa * sleepModifier * massClass_m
λ_m = ln(2) / t½_m
Fatigue_m(t) = min(100, Σ impact_{m,i} * e^(-λ_m * (t - t_i)))
Readiness_m = clamp(100 - Fatigue_m, 0, 100)
```

Kötü uyku yarılanma süresini uzatır, iyi uyku kısaltır. Kas kütle sınıfı da çarpandır: quadriceps bir ön kol kadar hızlı toparlanmaz.

MuscleInvolvementMatrix

232 satır

Hareket kalıbı × kas grubu katılım matrisi. Hem yorgunluk motorunun hem kuvvet merceğinin ortak anatomi kaynağıdır — iki ayrı "hangi kas çalışıyor" tablosu yoktur.

4.4 Mercek motorları

EnduranceEngine

227 satır

Her şey tek bir uydurmadan sarkar: kritik hız modeli. Birkaç en iyi çabadan sürdürülebilir bir hız ve onun üzerinde sonlu bir mesafe çıkarılır.

$$d = CS \cdot t + D'$$

Buradan varsayılan değil *ölçülmüş* tempo bölgeleri ve bu koşucuya kalibre edilmiş yarış tahminleri gelir. 120 saniyeden kısa çabalar D' tarafından domine edilir ve uydurmayı bükür; 2.400 saniyeden uzun çabalar iki parametrelili modelden sapar — ikisi de dışlanır.

Modelin bilinen zayıflığı gizlenmek yerine *söylenir*: oksijen alımının yavaş bileşeni için bir terimi yoktur, dolayısıyla uzun mesafe performansını fazla tahmin eder. Bu yüzden `RacePrediction` bir ekstrapolasyon katsayısı taşır ve maraton tahmini bunu ekranda açıkça belirtir.

Eğim düzeltmesi Minetti maliyet eğrisiyle yapılır; eğri minimumunu -20% civarında verir, ki bu ölçülmüş davranışla uyumludur ve testle sabitlenmiştir.

StrengthEngine

227 satır

Kaldırıcının gerçekten sorduğu üç soru: bar yukarı gidiyor mu, her kas yeterli iş alıyor mu, bir şey çarpık mı? Üçü de kaydedilen seanslardan ve 16 kaslık katılım matrisinden gelir — ikinci bir anatomi kaynağı eklemeyiz.

1RM	Epley ve Brzycki formülleri; düşük tekrarda birbirine yakın, yüksek tekrarda ayrışır ve bu ayrışma gösterilir
EŞİK	<code>setCountingThreshold = 0,25</code> — bir hareket kalıbı bir kası 25% 'in altında çalıştırıyorsa çalışan set sayılmaz
PENCERE	Hacim 7 gün, ilerleme geçmişi 180 gün

HybridEngine

237 satır

Motorun tek bir tezi var: **Hyrox'ta yarışı kaybettiren sayı, yorgunken koşarken ne kadar yavaşladığıdır**. İstasyon süreleri ölçülebilir ve herkes ölçüyor; kompanse koşu cezasını ölçen neredeyse yok. Motor tam olarak bunu hesaplar: taze koşu temposu ile istasyon sonrası koşu temposu arasındaki fark.

LongevityEngine

218 satır

Uzun ufuklu bileşik skor. Her sütun 0–100 puanlanır ve **her biri kullanıcının kendi geçmişine karşı** puanlanır — bir popülasyon tablosuna karşı değil. Mekanizma her yerde aynı: sinyalin kendi uzun penceresi içindeki yüzdelik konumu alınır, yüksek olan daha uygun olacak şekilde yönlendirilir, ve sütunun sahip olduğu sinyaller ortalılır.

Geçmişi yetersiz bir sinyal tahminle doldurulmaz — yokluğu kabul edilir ve sütunun ağırlığı yeniden dağıtılır.

4.5 Sağlık motorları

VitalsEngine

178 satır

18 hayati işaret için taban ve sapma. Bu motor `BaselineEngine`'i kasıtlı olarak *yeniden kullanmaz*: o, $\$4$ 'ün sözleşmesini birebir uygular — 60 günlük EWMA, 3σ 'da winsorize, metrik başına belgelenmiş öncül ve kalibrasyon kapısı. Bir hayati işaretin öncülü, şartname ağırlığı veya kalibrasyon gereksinimi yoktur; ihtiyacı olan kendi ufkundaki düz pencere ortalaması ve sapmasıdır.

İkisini karıştırmak ya öncülü olmayan on sekiz sinyal için öncül uydurmak ya da toparlanma tabanını gevşetmek anlamına gelirdi. $\$12$ gereği hiçbir okumanın "iyi" olup olmadığına karar verilmez; yalnızca kişinin kendi son aralığına göre nerede durduğu bildirilir.

StressEngine

208 satır

Günü böler. Zorlanma motorunun zaten okuduğu gün içi kalp hızı serisi ve günün antrenmanları üzerinden çalışır — yeni HealthKit yüzeyi yok, yeni depolama yok. Aynı veriye farklı bir soru: "ne kadar yük" değil, "yük nereden geldi ve gün ne zaman gevşedi".

§12: seans dışındaki sürekli yükselme *yük* ve *örüntü* olarak tarif edilir. Klinik anlamda strese asla stres denmez ve bu asla bir sağlık bulgusuna dönüşmez.

CycleEngine

250 satır

İki iş yapar: kaydedilen kanamadan bir günün hangi fazda olduğunu tahmin etmek, ve bir metriğin geçmişini faza göre bölerek bugünü kendisine benzeyen günlerle karşılaştırmak.

Asıl önemli olan ikincisi. Luteal fazdaki bir dinlenme kalp hızı, tüm döngü ortalamasına karşı karşılaştırıldığında her ay iki üç atım yüksek durur; bu farkı okuyan bir toparlanma motoru olmayan kötü bir sabah bildirir. Tabanı bölmek sistematik bir hatayı kaldırır — özellik eklemes.

§12: burada hiçbir şey gebelik çıkarmaz, doğurganlık tahmin etmez veya bir döngüyü düzenli/düzensiz diye nitelemez. Kullanıcının kaydettiğinden bir faz tahmin eder, ne kadar emin olduğunu söyler ve durur.

PainEngine

125 satır

Tek bir soru: kaydedilmiş bir ağrı girdisinden önceki günler, kaydedilmemiş bir günden öncekilerden daha mı ağırdı? Bu, uygulamanın zaten sahip olduğu iki şey arasında bir karşılaştırmadır — §12'nin çizdiği sınır tam olarak burasıdır. *Neden* sorusuna dair hiçbir şey bu sınırın dışındadır.

Pencere 48 saattir: gecikmeli kas ağrısı bir ile iki gün arasında tepe yapar.

LabInsightEngine

262 satır

Saklanan kan belirteçlerini gözleme çevirir. §12 çizgisi bu motorun üzerinde yaşadığı çizgidir ve açıkça söylenmeyi hak eder: **referans bandının dışındaki bir değer burada tam olarak tek bir çıktı üretir, o da "bunu hekiminize götürün".** Sebep yok, isim yok, takviye yok, doz yok, "bu genelde şu anlama gelir" yok.

Motorun gerçekten iyi olduğu şey, bir laboratuvar raporunun yapamadığıdır: kan alınmadan önceki günlerde kullanıcının ne yaptığını bilir ve panelin geri kalanının neye benzediğini bilir. Zenithium'un söylemeye hakkı olanlar: bir sayının nerede durduğu, nasıl hareket ettiği, ne kadar eski olduğu ve antrenman etrafında nasıl davrandığı.

4.6 Bağlam motorları

EnvironmentEngine

162 satır

Yol haritası üç şey saymıştı: gün ışığı, rakım ve saat dilimi kayması. İkisi yapıldı, üçüncüsü kasıtlı olarak yapılmadı.

Gün ışığı gerçek bir boşluğu doldurur: sirkadiyen motor uyanıklık eğrisini uyku zamanlamasına göre modeller, ama ışık baskın zeitgeber'dir — neredeyse tamamen kapalı mekânda geçen bir hafta o eğriyi *daha az* güvenilir kılar ve uygulama aynı kendinden emin yayı sunmak yerine bunu söylemelidir. **Saat dilimi kayması** ucuz ve gerçekten faydalıdır: gün kayıtları zaten hesaplandıkları dilimi taşır.

HeatAcclimationEngine

V4 · C7

198 satır

Sıcakta yapılan seanslardan ısı adaptasyonu. Literatürün desteklediği kısım zaman seyridir ve çalışmalar arasında tutarlıdır: etkinin çoğu günlük maruziyetin beş ilâ on günü içinde ortaya çıkar, plato kabaca iki haftada gelir, ve maruziyet olmadan iki ilâ üç haftada — kazanıldığından hızlı — söner.

Dolayısıyla model sınırlı bir birikim artı sönümdür, daha fazlası değil: eşğin üzerindeki her seans, ne kadar sıcak olduğu ve ne kadar sürdüğüyle ölçeklenen bir artış ekler; seanssız her gün biraz eksiltir. Sıcaklık ve nem HealthKit antrenman meta verisinden (`HKMetadataKeyWeatherTemperature`) okunur.

ReferenceNorms

V4 · C11

Domain/Norms

Yaş ve cinsiyet için $VO_2\text{max}$ referans yüzdeleri (FRIEND kayıt tablosu). Uygulamanın popülasyon karşılaştırması yaptığı tek yer; her yerde varsayılan hâlâ kişinin kendi geçmiştir.

ASSUMPTION NORM-1 — KONTROL EDİLDİ, TUTMADI

Tablo, dosyanın atıf verdiği kaynağa (Kaminsky ve ark. 2015) karşı kontrol edildi. Kontrol edilebilen dört hücrenin ikisi yanlış: 70–79 yaş 50. persentili erkekte 28,1 yerine **24,4**, kadında 21,5 yerine **18,3** olmalı. 20–29 hücreleri doğru.

Kalan 32 hücre bu makineden doğrulanamadı. Bu yüzden `isPublicationVerified` **false** ve ekran tablo aktarılana kadar karşılaştırma göstermiyor. Aritmetik ve testler duruyor; yanlış olan yalnızca 36 sayı.

4.7 Karar motorları

PrescriptionEngine

371 satır — en büyük motor

Her motorun cevabının bulunduğu yer. Karar tek yönde ilerler ve her adım bir öncekini daraltır:

1. **Toparlanma** niyeti belirler — itmek, korumak veya geri çekilmek.
2. **Yük oranı** ayarlar. Bir zirvenin tepesindeki yeşil sabah, daha fazla eklemek için izin değildir; iki kolay haftadan sonraki sarı sabah çoğu zaman izindir.
3. **Kas hazırlığı** seçenek eler. %40'taki bacaklara alt vücut işi yazmanın anlamı yoktur.

4. **Mercek** kalanı tanınabilir bir seansa çevirir. Aynı girdiler bir koşucu için "55 dakika kolay", sağlığını izleyen biri için "40 dakika tempolu yürüyüş" olur.

5. **Zorlanma tavanı** süreyi kırpar — **StrainEngine**'in integrali *tersine* koşturularak, böylece tahmin bu geceki sayıyla aynı ölçekte kalır.

Regl döngüsü bağlamı ayrı bir satır olarak eklenir ve kasıtlı olarak yumuşatılmaz:

```
static let phaseTypicalFraction: Double = 0.10

static func cycleContextLine(cycle: CycleContext?, band: RecoveryBand) ->
String? {
    guard let cycle, cycle.estimate.isConfident else { return nil }
    guard band != .green else { return nil }
    let q = cycle.estimate.qualifier // kısaltıldı
    guard let deviation = cycle.deviationFromPhaseMean else {
        return "\(q). Bu faz için henüz yeterli geçmiş yok."
    }
    guard abs(deviation) <= phaseTypicalFraction else {
        return "\(q). HRV'n bu fazın ortalamasından belirgin farklı."
    }
    return "\(q). Bu faz için olağan bir sabah."
}
```

Luteal fazı yumuşatma talebi hem kanıt gerekçesiyle (McNulty 2020 meta-analizi, faz etkisinin küçük ve tutarsız olduğunu bildirir) hem \$1 gerekçesiyle reddedildi; bir sonraki katkı sağlayıcının bunu geri getirmemesi için bir *değişmezlik testi* yazıldı.

PlanEngine

165 satır

Bugünün bir hedefe giden yolda nerede durduğu. Kasıtlı olarak küçük: faz sınırları olay tarihinden geriye sayılır ve konikleşmenin (taper) form üzerindeki etkisi yük motorunun zaten koşturduğu fitness/yorgunluk modeliyle hesaplanır — ikinci bir antrenman teorisi yok, program taklidi yok.

NarrativeEngine

251 satır

Deterministik anlatıcı. İnsanların bir sağlık uygulamasında "yapay zekâ" derken kastettikleri katman budur ve ne yaptığı konusunda kesin olmak gerekir: **bir modelden metin üretmez**. Bugün *hangi olgunun en çok önemli olduğuna* karar verir ve onu bir insanın söyleyeceği bir cümleyle söyler.

Her motor zaten katkı paylarını hesaplıyor; anlatıcının bütün işi onlara bakıp günü açıklayıcı seçmek. Her cihazda çalışır, model gerektirmez, pil harcamaz ve aynı sayılardan her seferinde aynı cümleyi üretir — yani test edilebilir. Dil modeli katmanı bunun *üstünde* oturur ve kararı asla değiştirmez.

CorrelationEngine

163 satır

Davranış–ölçüm karşılaştırması. Yaklaşım kasıtlı olarak basit ve savunulabilir: iki grup ortalaması, Welch standart hatasıyla bir güven aralığı ve Cohen'in *d*'si. Regresyon veya çok değişkenli model kurulmuyor — çünkü eldeki *n* bunu

taşımaz ve taşıyormuş gibi yapmak kullanıcıya hak etmediği bir kesinlik satmak olurdu.

ASSUMPTION CORR-1: güven aralığı normal yaklaşımı ($z = 1,96$) kullanır, t dağılımı değil. Gruplar 5'in altındaysa sonuç hiç üretilmez; 5–10 aralığında normal yaklaşım aralığı bir miktar dar tahmin eder. Telafi olarak minimum örneklem 6'da tutulur ve arayüz gözlem sayısını her zaman gösterir.

RacePlanEngine

V4 · C2

208 satır

Bir parkuru ve bir hedefi kilometre bölümlerine çevirir. Fikir şu: *koşucu tempo değil efor tutar*. Tepeli bir parkurda eşit *tempo* tutmak, tırmanışlarda inişlerden çok daha fazla harcamak ve ikinci yarıyı hiçbir şeyle koşarak bitişe varmak demektir. Eşit *efor* tutmak, temponun zeminle birlikte hareket etmesi demektir — planın işi bunun ne kadar olduğunu söylemektir.

`EnduranceEngine.gradeCostRatio` zaten "bu eğim ne kadara mal olur" sorusunu cevaplıyor; bu motor onu ters yönde koşturur: düz-eşdeğer tempo p verildiğinde, g eğiminde tutulacak tempo $p \cdot \text{ratio}(g)$ 'dir. Parkur profili GPX dosyasından okunur.

SessionShapeEngine

V4 · C8

125 satır

Yol haritası HealthKit antrenmanlarının hibrit ekranının modellediği Hyrox istasyonlarına sınıflandırılmasını istemişti. **Bu mümkün değil ve taklit edilmemeli:** bir antrenman özeti bir aktivite tipi, bir süre, bir mesafe ve bir ortalama kalp hızı taşır ve bunların hiçbirisi kızak itmeyi wall ball setinden ayırmaz. İkisi de "fonksiyonel kuvvet antrenmanı, 42 dakika" olarak gelir. Tahmin etmek birinin kaydını yapmadığı istasyonlarla doldururdu, ve yanlış bir kayıt boş bir kayıttan kötüdür.

Veride gerçekten *olan* şey tekrardır. Her salı sekiz kilometre koşan biri bunu sekiz kez kaydetmiştir; bunu bir şablon olarak geri sunmak — ön doldurulmuş süre, mesafe, yoğunluk — hiçbir şey uydurmadan yazma zahmetini kaldırır. Motor bunu yapar.

ARAYÜZ VE TASARIM SİSTEMİ

Arayüzün tamamı SwiftUI'dir: tek bir storyboard, tek bir `UIViewRepresentable` yoktur. Grafikler üçüncü parti bir kütüphaneden değil, Swift Charts'tan ve elle yazılmış `Canvas` çizimlerinden gelir.

5.1 Gezinme

Kök gezinme iOS 18'in değer tabanlı `Tab(value:)` API'siyle kurulmuştur. Beş sekme vardır ve ikincisi **merceğe göre değişir**:

SEKME	SİMGE	İÇERİK
Bugün	<code>heart.fill</code>	Toparlanma, reçete, günün özeti
Yük / Kaslar / Sağlık	merceğe göre	Dayanıklılık+hibrit → Yük · Kuvvet → Kaslar · Sağlık → Sağlık
Uyku	<code>moon.zzz.fill</code>	Uyku skoru, borç defteri, sosyal jetlag
Günlük	<code>square.and.pencil</code>	Özel giriş, korelasyon bulguları
Daha fazla	<code>square.grid.2x2.fill</code> 1	15 hedefli merkez (hub)

Merkez ekranı 15 hedef sunar: `strain · load · endurance · racePlan · strength · muscles · plan · vitals · hybrid · trends · bloodwork · report · documents · dataTransfer · settings`. Hepsi `HubDestination` enum'unda tanımlıdır ve `CaseIterable`'dir — yeni bir ekran eklemek, listeye bir `case` eklemektir.

5.2 Ekran envanteri

55 görünüm dosyası 19 aileye ayrılmıştır: `Today · Sleep · Journal · Load · Strain · Muscle · Strength · Hybrid · Endurance · Vitals · Trends · Bloodwork · Documents · Report · Plan · Circadian · Onboarding · Settings · DesignSystem`.

5.3 Renk altyapısı

Yol haritası v4'ün en fazla ölçüm gerektiren kararı buydu. Renkler `ZenithiumColor` üzerinden çağrılıyordu ve **712 çağrı yeri** vardı. Karanlık/aydınlık tema desteği eklemek için iki seçenek vardı:

SEÇENEK	SONUCU
Ortam (environment) enjeksiyonu	712 çağrı yerinin hepsi değişirdi; ayrıca 6 tanesi ortamı okuyamayan statik bağlamlarda (<code>WidgetStyle</code> , <code>ChartChrome</code> , <code>ZenithiumFont</code>) — bunlar hiç çalışmazdı
Varlık kataloğu	SEÇİLEN Renk çizim anında çözülür; hiçbir çağrı yeri değişmedi ; statik bağlamlar da doğru çalışır

Uygulanan yapı üç katmanlıdır:

```
// 1. Tek doğruluk kaynağı: bir Python tablosu
//   Scripts/generate-colors.py
PALETTE: dict[str, tuple[str, str, str]] # ad → (koyu, açık, açıklama)
    ↓ üretir
// 2. 19 .colorset klasörü + Swift enum
Resources/Zenithium.xcassets/Colors/*.colorset/Contents.json
Views/DesignSystem/ZenithiumColorAsset.swift // GENERATED
    ↓
// 3. Değişmeyen cephe
enum ZenithiumColor {
    private static func asset(_ a: ZenithiumColorAsset) -> Color { a.color }
    static var accent: Color { asset(.accent) } // 712 çağrı yeri aynı kaldı
}
```

Üretecin tek tabloda toplanması, dize anahtarlarını yazım hatasına kapalı hâle getirir: bir renk adı yalnızca bir yerde yazılır ve hem katalog hem enum ondan üretilir. `ZenithiumPalette` ayrıca `.dark` ve `.light` paletlerini gözden geçirilebilir değerler olarak tutar.

5.4 Tasarım sistemi bileşenleri

BİLEŞEN	GÖREVİ
ZenithiumFont	Tipografi ölçeği. Hepsi <code>Font.system(.style)</code> tabanlı — yani Dynamic Type 'i ücretsiz alır . Sabit punto yok.
ZenithiumMetrics	<code>ZenithiumSpacing</code> ve <code>ZenithiumRadius</code> . v4'te eklendi; dosyalara serpilmiş boşluk sabitlerinin yerini aldı.
ChartChrome	<code>.zenithiumChartChrome(dateAxis:desiredXCount:)</code> — her grafiğin eksen, ızgara ve etiket davranışını tek yerden verir.
SkeletonView	Yükleme iskeletleri (scored / cards / chart düzenleri). Parıltı yalnızca bloklara maskelenir ve Reduce Motion açıkken durur .
SeriesChartDescriptor	Yeniden kullanılabilir <code>AXChartDescriptorRepresentable</code> — grafikleri sesli grafik (audio graph) olarak erişilebilir yapar.
ArcGauge · Sparkline · SpectralField	<code>Canvas</code> ve <code>Path</code> ile çizilen özel görselleştirmeler. <code>Animatable</code> uyumlarıyla değer geçişleri yumuşatılır.
LensMark	Dört mercek için çizilmiş işaretler. v4'te SF Symbol yerine <code>Shape</code> geometrisi seçildi: yanlış kılavuzlu bir sembol gürültülü biçimde başarısız olmaz, sessizce kayık çizer — ve bu ortamda bunu doğrulayacak hiçbir şey yok.
BandChip · MetricTile · SectionCard · StateViews	Tekrar eden yerleşim öğeleri; boş/hata/yükleme durumları dâhil.

5.5 Erişilebilirlik

ÖZELLİK	UYGULANIŞI
Dynamic Type	Tüm tipografi <code>Font.system(.style)</code> ; özel ölçekli boyutlarda <code>@ScaledMetric</code>
Sesli grafik	<code>.accessibilityChartDescriptor</code> — v4'te 4 grafiğin 3'ünde eksikti, tamamlandı
VoiceOver rotor	Kas haritasında bölge bölge gezinme

ÖZELLİK	UYGULANIŞI
Reduce Motion	İskelet parıltısı ve geçiş animasyonları devre dışı
Dokunsal geri bildirim	<code>.sensoryFeedback</code> — bant değişimi, kayıt tamamlama
Sayısal geçiş	<code>.contentTransition(.numericText())</code> — değişen skor okunabilir biçimde döner

BİR DENETİM HATASININ DÜZELTİLMESİ

v4 yol haritası "11 görünüm dosyasında erişilebilirlik yok" diyordu. Bu, dar bir `grep`'ten geliyordu ve **yanlıştı**: erişilebilirlik ifadesi bulunmayan yalnızca 6 dosya var ve hiçbirisi görünüm değil; kas haritası rotoru zaten mevcuttu. Gerçek boşluk sesli grafiklerdi ve düzeltilen o oldu.

5.6 Tema

`AppearancePreference` üç değer alır: sistem, aydınlık, karanlık. Tip `Foundation`-only bir `Domain` dosyasındadır; SwiftUI köprüsü (`colorScheme` dönüşümü) ayrı bir dosyada durur. Bu bölünme keyfi değil — `Domain`'in SwiftUI'a bağımlı olması watch ve widget hedeflerini kırardı (bkz. §2.1).

SİSTEM ENTEGRASYONU

Uygulama telefonun içinde kalmaz: saatte canlı seans yürütür, kilit ekranında yaşayan bir kart gösterir, Kontrol Merkezi'ne bir düğme koyar ve Siri'ye cevap verir. Hepsi tek bir veri modelini paylaşır.

6.1 Apple Watch

watchOS uygulaması 7 dosyadır ve tek bir işi vardır: seansı yürütmek. `HKWorkoutSession` ve `HKLiveWorkoutBuilder` ile kalp hızı ve enerji akışını okur, `LiveSessionEngine` ile anlık zorlanmayı hesaplar ve durumu telefona yayınlar.

NEDEN DOYUM TUZAĞI ÖNEMLİYDİ

Canlı zorlanma hesabı naif yapıldığında yanlış çıkıyor. Zorlanma ölçeği doyuma giden bir üstel eğridir; seansın parçalarını ayrı ayrı zorlanmaya çevirip toplamak, tek integralden farklı bir sayı verir. Ölçülen fark: naif toplama 13,4 verirken doğru integral 11,6 veriyordu. Motor TRIMP'i biriktirir ve *ölçeğe yalnızca en sonda* çevirir.

WATCHCONNECTIVITY

Telefon-saat köprüsü `WCSession.updateApplicationContext` kullanır — mesaj kuyruğu değil. Bu bilinçli: uygulama bağlamı **son-durum-kazanır** semantiğine sahiptir, yani saat üç saniye boyunca bağlantısız kalırsa üç ara güncelleme birikmez, yalnızca en sonuncusu teslim edilir. Canlı bir seans için doğru davranış budur; sıraya alınmış eski durumlar kullanıcıya geçmişini gösterirdi.

6.2 Widget'lar

ZENITHIUMWIDGETBUNDLE — 7 WIDGET + LIVE ACTIVITY

WIDGET	AİLE	GÖSTERDİĞİ
RecoveryStrainWidget	Ana ekran	Toparlanma + zorlanma çifti
ThreeDayTrendWidget	Ana ekran	Üç günlük eğilim
JournalWidget	Ana ekran	Günlük girişi kısayolu
RecoveryCircularWidget	Kilit — dairesel	Toparlanma halkası
RecoveryRectangularWidget	Kilit — dikdörtgen	Skor + tek satır gerekçe
RecoveryInlineWidget	Kilit — satır içi	Tek satır metin
RecoveryControlWidget	Kontrol Merkezi	Eylem düğmesi kısayolu
LiveSessionActivity	Live Activity	Saatte koşan seans — kilit ekranı + Dynamic Island

V4'TE BULUNAN SESSİZ HATA

`JournalWidget` ve `RecoveryControlWidget` Faz 22'de yazılmış ama `WidgetBundle` 'a hiç eklenmemişti. Kodu ne kadar doğru olursa olsun, pakette olmayan bir widget cihazda var değildir — ikisi de aylardır ölüydü. v4'te kayıt edildiler.

6.3 Live Activity

`ActivityKit` ile saatte koşan seans kilit ekranında ve Dynamic Island'da canlı görünür. `ActivityAttributes` sabit bilgiyi (seans tipi, başlangıç), `ContentState` değişen bilgiyi (süre, kalp hızı, zorlanma) taşır. Güncellemeler `ActivityContent(staleDate:)` ile yayımlanır — böylece saat bağlantısını kaybederse kart "bayat" işaretlenir ve kullanıcıya donmuş bir sayı gösterilmez.

6.4 App Intents ve arka plan

YÜZEY	DAVRANIŞ
AppIntents	Siri ve Kısayollar: toparlanmayı sor, günlük girişi aç, seans başlat
BackgroundTasks	<code>BGAppRefreshTask</code> ile sabah yeniden hesaplaması; <code>DispatchQueue</code> yasağı nedeniyle kayıt <code>Task</code> ile yapılır
OSSignposter	v4'te eklendi — <code>orchestration</code> , <code>labs</code> , <code>ui</code> kategorileri; Instruments'ta ölçülebilir aralıklar

ZEKÂ KATMANI

Uygulamada "yapay zekâ" iki farklı anlama gelir ve ikisini karıştırmamak önemlidir. Kararı veren şey deterministik motorlardır; dil modeli yalnızca o kararı yeniden ifade eder ve asla değiştiremez.

7.1 Üç katmanlı anlatım

```
1. NarrativeEngine      Hangi olgu bugün önemli? → deterministik cümle
                        Her cihazda çalışır · test edilebilir · pil harcamaz
                        ↓
2. FoundationModelNarrator Aynı cümleyi daha doğal ifade et (cihaz içi model)
                        Yalnızca modeli olan cihazlarda · #if
                        canImport(FoundationModels)
                        ↓
3. SafetyFilter          Çıktı $1 ve $12'yi çiğniyor mu? Çiğniyorsa reddet
                        Reddedilirse 1. adımın cümlesi gösterilir
```

Bu sıralamanın sonucu şudur: **dil modeli başarısız olabilir, garanti olamaz.** Model yoksa, yavaşsa, saçmalarsa veya filtreye takılırsa kullanıcı yine doğru bir cümle görür — eksik bir şey olmaz. Çoğu cihazda 2. adım hiç çalışmaz ve bu, kullanıcı için fark edilmez.

7.2 Güvenlik filtresi

Üretilen her cümlemin geçtiği son nokta. Uyguladığı üç çizgi doğrudan şartnameden gelir:

- **\$12** — teşhis yok, bir duruma isim verme yok, tedavi yok, doz yok.
- **\$1** — kalori hedefi yok, kilo hedefi yok, kısıtlama teşviki yok.
- **\$12** — kullanıcıya asla bir belirtiyi görmezden gelmesi söylenmez.

Eşleştirme *normalize edilmiş token'lar üzerinde önek kökleriyle* yapılır, çünkü Türkçe eklemelidir: "ilaç", "ilacı" ve "ilaçları" tek bir fikir ve üç token'dır. Dört karakter ve üzeri bir kök önekle eşleşir; daha kısası birebir eşleşmek zorundadır — bu, "doz" kökünün sadece o harflerle başlayan bir kelimede tetiklenmesini önler.

KELİME DEĞİL İDDİA YASAKTIR

Liste, uygulamanın kendi dürüst metninde geçen kelimeler konusunda kasıtlı olarak muhafazakârdır. "Hastalık" kullanıcının kendi kaydettiği bir günlük davranışdır, dolayısıyla bir cümle onu meşru biçimde adlandırabilir. Yasak olan *iddiadır* — "hastasın", "hastalığın var" — kelimenin kendisi değil.

Yanlış pozitifler burada zaten ucuzdur: geri düşüş deterministik metindir ve o metin her hâlükârda doğru olacaktı.

7.3 Laboratuvar OCR hattı

Kullanıcı bir kan tahlili PDF'i eklediğinde hat şöyle işler:

ADIM	ÇATI	İŞ
1	PDFKit	Gömülü metin katmanı varsa doğrudan çıkarılır
2	Vision	Metin katmanı yoksa (taranmış PDF) OCR çalıştırılır
3	LabDocumentReader	Satırlara ayırma, sayı ve birim ayrıştırma
4	BiomarkerDefinition	Satırdaki adı katalogla eşleştirme
5	SwiftData	<code>BloodMarker</code> kaydı; PDF <code>.externalStorage</code> ile saklanır

BİYOBELİRTEÇ KATALOĞU

51	202	9	1.353
BİYOBELİRTEÇ	EŞ AD	PANEL	SATIR

Paneller: hematoloji, lipid, metabolik, tiroid, hormon, iltihap, demir, mikrobesein, organ. Her belirteç `SexSpecificRange` taşır — transaminazlar gibi referans bandı cinsiyete göre değişen belirteçlerde herkese tek bir bant göstermek yanlış olurdu.

EŞLEŞTİRME PERFORMANSI

İlk uygulama her satır için 202 eş adın hepsini deniyordu ve her deneme yeni bir dize kopyası üretiyordu. v4'te bu, **token indeksli** bir yapıya çevrildi:

```
struct SynonymNeedle { /* eş ad, token'ları, hedef belirteç */ }

static let needles: [SynonymNeedle] // 202 eş ad
static let needleIndex: [String: [SynonymNeedle]] // ilk token → adaylar
// uzundan kısaya sıralı

// Sekiz token'lı bir satır artık 202 değil, bir avuç aday değerlendirir.
guard let candidates = needleIndex[tokenTexts[position]] else { continue }
```

Eşleştirme *kopyalamadan* yapılır — `matches(_in:at:)` token dizisi üzerinde konum karşılaştırır, alt dize üretmez. Uzundan kısaya sıralama, "ldl kolesterol"ün "ldl"den önce denenmesini garanti eder.

`candidatesExamined(inLine:)` yalnızca testler içindir ve `BiomarkerCatalogTests` aday sayısındaki düşüşü doğrular.

VERİ TAŞINABİLİRLİĞİ

Sunucusu olmayan bir uygulamada veri kullanıcının cihazında hapis olmamalıdır. Zenithium dört çıkış yolu sunar ve hepsi kullanıcının başlattığı, kullanıcının hedefini seçtiği işlemlerdir.

8.1 Tam arşiv

`ZenithiumArchive` her `SwiftData` modelini tek bir `Codable` yapıya toplar; `ArchiveService` onu bir dosyaya yazar ve geri okur. Kullanıcı arayüzü `DataTransferView`'dur.

İKİ KEZ TEKRARLANAN HATA – KAYDA GEÇİRİLDİ

Swift'in sentezlediği `Decodable`, **eksik bir anahtarda özellik varsayılanına düşmez — hata fırlatır**. Bu iki kez vurdu: önce `supplementCourses` alanında (`Optional` yapılarak çözüldü), sonra çekirdek bir tip olan `UserProfileSnapshot.appearance`'da. İkincisi için tüm `Codable` uyumu elle yazıldı ve sonradan eklenen her alan `decodeIfPresent` ile okunuyor.

Her ikisi de elle yazılmış bir fikstür testiyle yakalandı: eski sürümde üretilmiş bir arşiv JSON'u, yeni kodla okunabilmelidir. Bu test olmasaydı, kullanıcılar eski yedeklerini geri yükleyemeyecekti ve bu ancak sahada fark edilirdi.

8.2 Hekim raporu

`ReportDocumentRenderer` son 90 günün özetini bir PDF'e çizer: taban çizgileri, eğilimler, kan belirteçleri, ağrı kayıtları. Rapor §12'ye uyar — bulgu bildirmez, veri sunar ve yorumu hekime bırakır.

Bu, depodaki **tek `UIKit` import'udur**. Gerekçe dosyada yazılıdır: `UIGraphicsPDFRenderer`'ın `SwiftUI` karşılığı yoktur ve saf `Core Graphics` ile aynı işi yapmak metin yerleşimini elle çözmeyi gerektirirdi.

8.3 GPX ve yarış planı

`GPXReader` bir parkur dosyasından mesafe ve yükseklik profili çıkarır; `CourseProfile` bunu `RacePlanEngine`'in okuyabileceği bölümlere çevirir. Üretilen plan `RacePlanIO` ile dışa aktarılabilir. Ayırıştırıcı `XMLParser` tabanlıdır ve bozuk/eksik yükseklik verisine karşı testlidir.

8.4 Belge kasası

`HealthDocumentLog` kullanıcının eklediği sağlık belgelerini saklar. PDF'in kendisi `@Attribute(.externalStorage)` ile veritabanı

dosyasının dışında tutulur — aksi hâlde birkaç megabaytlık taramalar SwiftData deposunu şişirirdi.

Arama, çıkarılmış metin üzerinde çalışır. v4'ten önce her tuş vuruşunda ~2 MB metin yeniden katlanıyordu; şimdi `DocumentsViewModel.Content` yükleme anında bir `searchText: [UUID: String]` indeksi ve `byYear` gruplamasını *bir kez* kurar ve filtreleme yalnızca indekse bakar.

TEKNOLOJİ YIĞINI

Yığında yalnızca Apple'ın birinci parti çatıları var. Swift Package Manager bağımlılık listesi boştur ve bu bir ideoloji değil, tedarik zinciri yüzeyini sıfıra indirme kararıdır.

KULLANILAN ÇATILAR – İMPORT SAYIMI

ÇATI	IMPORT	NEREDE
Foundation	169	Her yerde; <code>Domain</code> 'in tek bağımlılığı
SwiftUI	71	Tüm arayüz, tüm çizim
Observation	21	<code>@Observable</code> ViewModel'ler
SwiftData	20	Kalıcılık, şema, göç
WidgetKit	9	7 widget + Control Widget
Charts	7	Eğilim ve dağılım grafikleri
HealthKit	5	Okuma hattı, canlı seans
UniformTypeIdentifiers	4	Dosya içe/dışa aktarım tipleri
AppIntents	3	Siri, Kısayollar, Eylem düğmesi
ActivityKit	3	Live Activity, Dynamic Island
WatchConnectivity	2	Telefon-saat köprüsü
OSLog	2	Günlükleme ve <code>OSSignposter</code>
Accessibility	2	<code>AXChartDescriptor</code> — sesli grafikler
WatchKit	1	watchOS yaşam döngüsü
Vision	1	Taranmış tahlil PDF'lerinde OCR
PDFKit	1	Gömülü PDF metni çıkarma
FoundationModels	1	Cihaz içi dil modeli (koşullu derleme)
CoreGraphics	1	Özel çizim yardımcıları
BackgroundTasks	1	Sabah yeniden hesaplaması
UIKit	2	<i>İki istisna</i> — <code>ReportDocumentRenderer</code> 'da PDF çizimi, <code>ProtectedDataGuard</code> 'da veri koruma durumu. İkisinin de birinci parti karşılığı yok

9.1 Dil ve araçlar

ÖGE	SÜRÜM / DURUM
Swift dil kipi	6 — sıkı eşzamanlılık <i>complete</i>
Hedef platform	iOS 18 · watchOS 11
Test çatısı	Swift Testing (<code>@Test</code> , <code>#expect</code> , <code>#require</code>)
Proje üretimi	XcodeGen (<code>project.yml</code>) + <code>Scripts/generate-project.py</code>

ÖĞE	SÜRÜM / DURUM
Varlık üretimi	Scripts/generate-colors.py
Bağımlılık yöneticisi	Yok — SPM listesi boş

KALİTE VE DOĞRULAMA

588	66	10.2K	106
TEST	TEST PAKETİ	TEST SATIRI	KAYITLI VARSAYIM

10.1 Test stratejisi

Motorlar saf olduğu için testler de saftır: cihaz yok, veritabanı yok, zaman yok. Dört tür test kullanılıyor.

TÜR	NE YAPAR
Altın değer (golden)	Bilinen bir girdi için bilinen bir çıktı. <code>StrainEngineTests</code> beş TRIMP çipasını $\pm 0,1$ içinde doğrular; bu sayılar değişirse test kırılır.
Değişmezlik (invariance)	Bir davranışın <i>değişmemesini</i> sabitler. Örnek: luteal fazın reçeteyi yumuşatmadığını doğrulayan test — gelecekteki bir katkı sağlayıcı bunu iyi niyetle geri getirirse test kırılır.
Parametrelili	<code>@Test(arguments:)</code> ile bir mantığın tüm enum durumları üzerinde koşturulması
Fikstür	Elle yazılmış bir JSON arşivi; geriye dönük uyumluluğu doğrular. İki <code>Decodable</code> hatasını yakalayan test budur.

48 TEST DOSYASI

AccessibilityCopyTests · AppearanceTests · BaselineEngineTests ·
 BiomarkerCatalogTests · BloodworkTimelineTests · BoundedConcurrencyTests ·
 CircadianEngineTests · CorrelationEngineTests · CycleEngineTests ·
 DayRecordCacheTests · EnduranceEngineTests · FatigueEngineTests ·
 GPXReaderTests · HeatAcclimationEngineTests · HybridEngineTests ·
 InvolvementMatrixTests · LabInsightEngineTests · LabReportParserTests ·
 LabTableDetectionTests · LensMarkTests · LiveSessionEngineTests ·
 LiveSessionOutboxTests · LiveSessionSnapshotTests ·
 LiveSessionTrackTests · LongevityEngineTests · NumberFormattingTests ·
 PainEngineTests · PendingJournalTests · PerformanceRegressionTests ·
 PipelineIntegrationTests · PrescriptionEngineTests ·
 ProtectedDataGuardTests · RacePlanEngineTests · RecoveryEngineTests ·
 ReferenceNormsTests · SafetyFilterTests · ScientificBoundaryTests ·
 SessionShapeEngineTests · SleepDebtEngineTests · SleepScoreEngineTests ·
 StrainEngineTests · StrengthEngineTests · SupplementCorrelationTests ·
 TrainingLoadEngineTests · VitalsEngineTests · WidgetRefreshTests ·
 ZenithiumArchiveTests · ZenithiumErrorTests

10.2 Derleme dışı doğrulama

Swift derleyicisi bulunmadığı için üç ikame yöntem kullanıldı ve her biri gerçek hata yakaladı.

YÖNTEM	YAKALADIĞI
Python'da yeniden hesap	Her yeni motorun matematiği bağımsız olarak Python'da kuruldu ve beklenen değerlerle karşılaştırıldı. Yakaladıkları: canlı seans doyum tuzağı (naif 13,4 vs doğru 11,6); hibrit mercek işareti yüksekliğinin 6,94 < gereken 7,2 olması; yarı maraton bölüm sayısının 21 olması (22 değil); eğitim tolerans hatası (365 vs 365,25 günlük yıl).
Statik kaynak-kapanış taraması	Watch ve widget hedeflerinin derleyeceği dosya kümesi elle çıkarılıp her yığından sonra tarandı. Yakaladıkları: iki adet <code>Domain</code> → <code>Engines</code> sızıntısı (§2.1), her ikisi de iOS'ta görünmez olurdu.
Dosya varlık kontrolü	Heredoc ile yazılan dosyalar için <code>ls</code> doğrulaması. Yakaladığı: <code>mkdir -p</code> öncesi yazılan üç dosyanın sessizce kaybolması.
Sembol çözümleyici (<code>check-symbols.py</code>)	Dört hedefin her birinde, kaynakta geçen her tip adının o hedefin derleyeceği dosyalarda ya da bilinen bir çerçevede tanımlı olduğunu doğrular; ayrıca adlandırılmış argüman etiketlerini, yasaklı yapıları ve süslü parantez dengesini tarar. Yakaladıkları: <code>ReportView.swift</code> 'in <code>UIActivityViewController</code> kullanıp <code>UIKit</code> 'i içe aktarmaması — dosya yalnızca tesadüfen derleniyordu; iki ayrı dosyada aynı adı taşıyan iki <code>ShareLinkSheet</code> .

YİNE DE DERLENMEDİ

Bu yöntemler matematik hatalarını ve mimari ihlalleri yakalar.

Sözdizimi hatalarını, tip uyumsuzluklarını ve eşzamanlılık ihlallerini yakalamaz. İlk `xcodebuild`'de hata çıkması normaldir ve hatalar geri bildirildikçe düzeltilecektir.

10.3 Varsayım kayıt defteri

Belgelenmiş bir cevabı olmayan her karar bir *varsayım* olarak numaralandırıldı ve `docs/PHASE0-ASSUMPTIONS.md`'ye yazıldı. Kodda ilgili yerde numarasıyla anılır — böylece bir kararın neden öyle verildiği kaynak dosyanın kendisinden okunabilir.

106 VARSAYIM — EN KALABALIK KATEGORİLER

ÖNEK	ADET	ALAN
UI	9	Arayüz davranışı, yerleşim kararları
API	9	Apple API'lerinin belgelenmemiş davranışı
SLEEP	6	Uyku evresi ve süre yorumu
VM • STORE • LAB	5 • 5 • 5	ViewModel, kalıcılık, laboratuvar
MUSCLE • BUILD • BASE • AI	4 • 4 • 4	Anatomi, proje kurulumu, taban çizgisi, model
Diğer 30 önek	45	WIDGET, WATCH, VAULT, CIRC, BG, VITAL, TEST, STRAIN, RX, REPORT, PAIN, LOAD, HRMAX, CYCLE, CONST, ARCH, ZONE, SAFE, RPE, RECOV, PLAN, ORCH, NAV, NAME, MOCK, LOG, LOC, INTENT, DESIGN, DAY

İki varsayım kontrol edildi ve **ikisi de tutmadı**. **NORM-1:** FRIEND tablosu kaynağa karşı kontrol edildi, kontrol edilebilen dört hücrenin ikisi yanlış çıktı; sayılar tahmin edilmek yerine silindi ve özellik doğrulanana kadar kapatıldı (bkz. §4.6). **BUILD-1:** `project.yml`'in

yetkili olduğu iddiası *dört* kez yanlış çıktı — önce watch hedefi eksikti, sonra widget hedefinde `PendingJournalStore.swift`, sonra `MARKETING_VERSION` hiç tanımlanmamıştı (saat uygulaması boş bir `CFBundleShortVersionString` ile paketleniyordu, ki App Store Connect bunu doğrudan reddeder), ve son olarak üreteç `developmentRegion = en` yazarken `project.yml` `tr` diyordu. Dördü de düzeltildi ve `Scripts/check-target-sources.py` her birinin tekrarını engelliyor.

10.4 Yayın öncesi tarama — sürüm 0.1

Sürüm 0.1'e giden son geçiş, yeni özellik yazmadan yalnızca mevcut kodu tarayan dört adımdı. Bulunanların ortak bir şekli var: **hiçbiri derlemeyi bozmuyordu**. Her biri çalışan, testten geçen, sessizce yanlış davranan koddu — yayın öncesi taramanın var olma sebebi tam olarak bu sınıf.

YAYIN TARAMASININ BULGULARI

BULGU	SESSİZCE NE YAPIYORDU
Kirli veri sızıntısı 3 çağrı yeri	Eş zamanlı okuyan üç yer — temel seri, gecelik ortalamalar, hayati işaretler ekranı — "bu hata tek bir ölçümü mü yoksa tüm cevabı mı düşürür" sorusuna kendi elle tutulan listeleriyle cevap veriyordu. <code>healthDataProtected</code> kilitli cihaz yolu eklendiğinde üçünün de dışında kaldı, dolayısıyla kilitli bir telefonda sabah hesabı şifreli kalan ölçümleri düşürüp <i>kalanlardan</i> bir toparlanma skoru üretiyordu. Şimdi tek bir <code>ZenithiumError.blocksPartialResults</code> var ve üç yer de ona bakıyor.
Yutulan iptal	<code>Task.checkCancellation()</code> <code>CancellationError</code> fırlatır, <code>ZenithiumError.cancelled</code> değil. Genel <code>catch</code> dalları bunu "okunamayan ölçüm" sayıyordu, yani terk edilmiş bir okuma yine de ekran çiziyordu.
Gözlemci yarışı	<code>HealthObservationRelay.start()</code> yuvayı iki <code>await</code> 'ten sonra dolduruyordu. Bir aktör <code>await</code> boyunca yalıtımını bırakır; iki eş zamanlı çağrı ikisi de <code>nil</code> kontrolünden geçip iki akış açıyor, ikinci atama birincisini öksüz bırakıyordu — ve <code>stop()</code> öksüzü iptal edemiyordu.
Sürüm dağılıklığı	Sürüm dört yerde yaşıyor ve kendisiyle çelişiyordu: iki <code>Info.plist</code> <code>1.0</code> sabitliyor, saat <code>\$(MARKETING_VERSION)</code> okuyor, <code>project.yml</code> o değişkeni hiç tanımlamıyordu.
Doğrulanmamış norm tablosu	FRIEND VO ₂ maks tablosu "yaygın olarak tablolandığı hâliyle" aktarılmıştı. Kontrol edilebilen dört hücrenin ikisi yanlıştı ve fark yuvarlama değildi (70–79 erkek ortancası: tabloda 28,1, yayında 24,4).
Yorum kayması	Minetti eğrisinin yorumu dönüm noktasını -10% diye anlatıyordu; katsayılardan ölçülen değer $-18,1$. Yorum düzeltildi ve minimum <code>MinettiCurveTests</code> ile çivilendi.
Denetlenmeyen eşleşmeler	Saatin eşlik ettiği paket kimliği, arka plan görev kimliği ve geliştirme dili — üçü de iki yerde yazılıp hiçbir yerde karşılaştırılmıyordu. Üçü de yanlışken çökme değil sessizlik üretir.
Denetlenmeyen kaçış	Uygulamadaki tek <code>nonisolated(unsafe)</code> , derleyicinin doğrulayamadığı bir savla ("tek bir enum ataması yarışamayacak kadar küçük bir pencere") ayakta duruyordu. Kaçındığı aktör sıçraması bedavaydı; kaldırıldı.

İKİNCİ TUR — ARAYÜZ DİLİ, SAYI YAZIMI, SAAT

İlk tur mimariye ve hata yönetimine bakmıştı. İkinci tur ekrana baktı ve daha fazlasını buldu, çünkü baktığı yer daha önce hiç bakılmamış olan verdi.

BULGU

SESSİZCE NE YAPIYORDU

Ondalık ayırıcı
her sayı

`String(format:)` C'nin biçimlendiricisidir ve cihazın bölgesi ne olursa olsun nokta yazar. Uygulamadaki *her* ondalık "12.3" olarak çiziliyordu — kendi metinlerinde "12,4" yazan ve laboratuvar ayırıştırıcısı tam da Türk raporları virgül bastığı için virgüllü biçimi kabul etmek üzere yazılmış bir arayüzde. Aynı hatanın aynadaki hâli yüzde işaretindeydi: `34%`, Türkçenin yazımı `%34`. Kuralı açıklayan not hemen yanındaki fonksiyonun başında duruyordu.

Sesli metinler
~40 dize

`TrainingLens.accessibilityName` tablosunun tamamı ve uygulama, widget, saat genelinde otuza yakın etiket İngilizceydi. Hiçbir şey yakalamadı çünkü hiçbir şey erişilebilirlik metnine bakmıyor: ekranlar doğru okunuyordu ve bunu duyacak tek kişi Türkçe konuşan bir VoiceOver kullanıcısıydı.

Görünen metinler
9 cümle

Dokuz tam İngilizce cümle ekranda duruyordu. Hepsı koşullu: yalnızca toparlanmanın düşük olduğu bir günde, bir sensörün eksik kaldığı bir gecede, nabız verisinde boşluk olan bir günde çiziliyorlar. O günlerden birinde ekrana kimse bakmamıştı.

Saatte duraklatma

Süre `Date().timeIntervalSince(startedAt)` ile ölçülüyordu — duvar saati, duraklamayı da koşuyormuş gibi sayar. Devam edince ekrandaki süre duraklama kadar sıcıyor, her örneğin damgası aynı sıçramayı taşıyor, ve tavana varış projeksiyonu kişinin durduğu süreyle seyreltilmiş bir hız okuyordu. İmpuls integrali zaten güvendeydi: uzun bir boşluk hiçbir katkı yapmıyor. Bozuk olan etrafındaki saatti.

Yedek aktarım
193 KB

WatchConnectivity bir yükü 65.536 baytla sınırlar ve aşımı uygulamanın yazmadığı bir delegate geri çağırısıyla bildirir — yani sessizce. Kodlama örnek başına 53 bayt, 3.600'lük tavan 193 KB ediyordu: yirmi dakikadan uzun her kopukluk *hiçbir şey* teslim etmiyordu, ki kutu tam olarak o kopukluk için var.

Sonlu olmayan sayılar

`String(format:)` `nan` ve `inf` basar. İkisi de erişilebilirdi — boş bir pencere üzerinden bir oran bir bölme uzaklıkta — ve ikisi de ekrana o kelimelerle çıkardı.

Üç bölünmüş renk

Canlı seans bandının rengi üç yüzeyde çiziliyor ama iki yerde tanımlıydı. İkisi birbiriyle uyumluydu; saatle uyumlu kalmalarını sağlayan hiçbir şey yoktu.

DENETLEYİCİNİN KENDİSİ DE YANLIŞTI

Arayüz dili denetimi ilk iki koşusunda "temiz" dedi ve yanılıyordu: paylaştığı yardımcı fonksiyon her dize sabitini boşaltıyordu, yani tarama **içinde hiç metin olmayan bir kaynağa** bakıyordu. İkinci hata, kelimeleri ASCII'ye göre ayırıp `band1`'yı `band`'a bölmesi ve uygulamanın kendi Türkçesini İngilizce sanmasıydı. İkisi de negatif testle bulundu — bir denetim, kasıtlı olarak bozulmuş bir girdide düşmüyorsa denetim değildir.

NE BULUNMADI

Bu tarama ad çözümlemesini, yapılandırma tutarlılığını ve mantık sızıntılarını kapsar. **Tip çıkarımını, aşırı yükleme çözümlemesini, jenerik kısıtları ve Swift 6 yalıtım kurallarını kapsamaz** — bunlar yalnızca derleyicinin görebileceği şeylerdir ve bu kod hâlâ derlenmedi.

YOL HARİTASI V4

Faz 32 tamamlandıktan sonra üç başlıkta bir dalga daha yapıldı: performans, tasarımın kusursuzlaştırılması ve yeni özellikler. Yol haritası genel tavsiyelerden değil, **kod taramasından** çıkarıldı — her madde bir dosyayı ve o dosyadaki somut durumu gösteriyordu.

21	171	+12.979	-742
COMMIT	DOSYA	EKLENEN SATIR	SİLİNEEN SATIR

11.1 A — Performans

#	MADDE	ÖLÇÜLEN SONUÇ
A9	Ölçüm	<code>ZenithiumSignpost</code> üç yolda; <code>PerformanceRegressionTests</code> iş sayısı üzerinden doğruluyor. İlk yapılan madde bu oldu — ölçmeden optimize etmemek için.
A1	Kas haritası	Boyut sabitken kare başına spline üretimi 0 (önce düzen başına 42). <code>BodyPathCache</code> yolları (boyut, taraf) başına belleğe alıyor.
A2	Biyobelirteç	Satır başına normalizasyon 0 (önce 252); aday tarama 252 → 2-9
A3	Hayati işaretler	18 ardışık okuma → 4 eşzamanlı görevli sınırlı grup
A4	Gün kayıtları	<code>DayRecordCache</code> ; Bugün yenilemesi 4 okuma → 1
A5	Paylaşılan pencere	Kuvvet ve yük ekranları aynı 120 günlük okumayı paylaşıyor
A6	Belge araması	Tuş başına 2 MB katlama → yükleme başına bir kez
A7	HealthKit	İki döngü içi bekleme → görev grubu
A8	Taban çizgileri	Doldurma geçişinde 8 özdeş 60 günlük okuma → 1

11.2 B — Tasarım

#	MADDE	SONUÇ
B1	Boşluk ölçeği	<code>ZenithiumMetrics</code> ; 392 çağrı yeri belirtece taşındı
B2	Grafik çerçevesi	<code>.zenithiumChartChrome()</code> ; dört kopya → bir tanım
B3	Hareket	Yay girişte 0'dan yazılıyor; değişen her sayı <code>numericText</code>
B4	Dokunsal	2 → 6 yer, yalnızca durum değiştiren eylemlerde
B5	Yükleme	Dönen çark → ekranın iskeleti
B6	Palet katmanı	Aydınlık palet yazıldı; 19 renk tek üretilen dosyada; 712 çağrı yerinin hiçbirini değişmedi
B7	Dynamic Type	Korumasız üç etiket düzeltildi
B8	Erişilebilirlik	Dört zaman serisi grafiğinin dördü de artık çalınabilir
B9	Simge dili	<code>LensMark</code> — SF Symbol yerine <code>Shape</code> geometrisi, dört testle doğrulanıyor

#	MADDE	SONUÇ
B10	Saat/widget boşluk	Ortak ölççeğe taşındı

11.3 C — Yeni özellikler

#	ÖZELLİK	SONUÇ
C1	Canlı seans	Saatte, <code>LiveSessionEngine</code> üzerinden
C2	Yarış temposu	GPX → kilometre hedefleri, Minetti eğrisi tersine
C3	Tahlil zaman çizelgesi	Panel gruplaması + yıllık değişim hızı
C4	Uyku defteri	14 günlük yuvarlanan borç + sosyal jetlag; fazla uyku yarım geri ödüyor
C5	Takviye korelasyonu	Kür = pencere; mevcut istatistiğe <code>CorrelationSubject</code> olarak giriyor
C6	Faza duyarlı reçete	Faz seansı değiştirmiyor , okumayı açıklıyor — testle sabitlendi
C7	Isı adaptasyonu	Seans hava verisinden; 7 gün %76, 14 gün %94, 3 hafta ara %13
C8	Oturum tanıma	Tekrar eden seans şekilleri; istasyon tahmini bilerek yapılmadı
C9	Veri taşıma	Sürümlenmiş <code>.zenithium</code> arşivi, dışa + içe
C10	Widget ailesi	Kilit ekranı üç aile + Live Activity; kayıtsız iki widget bulundu
C11	Referans bantları	VO ₂ max, FRIEND kayıtları; yaş/cinsiyet yoksa karşılaştırma yok

11.4 Denetimin yanıldığı yedi yer

Yol haritasını ben yazdım ve uygulamaya geçince yedi noktada yanlış olduğu ortaya çıktı. Bunları kayda geçirmek, düzeltmek kadar önemli.

MADDE	İDDİA	GERÇEK
B8	"11 görünüm dosyasında erişilebilirlik yok"	Dar bir <code>grep</code> 'ten geliyordu. Etiketsiz altı dosya var ve hiçbirisi görünüm değil; kas haritası rotoru zaten mevcuttu. Gerçek boşluk sesli grafiklerdi.
A8	"60 günlük doldurma = 60 okuma dalgası"	Doldurma zaten geçiş başına 7 günle sınırlıydı. Gerçek israf, bir geçişteki her günün <code>refreshBaselines</code> 'ı aynı <code>now</code> ile çağırması — 8 özdeş okuma ve 8 özdeş yazma.
A6	"SwiftData izdüşüm okuması yok"	Doğru ama önemsiz. Gerçek maliyet <code>matches(query:)</code> 'ydi: her tuşta, hiç değişmeyen bir sonuç için 2 MB katlama.
C10	"Kilit ekranı widget'ları eklenmeli"	Yarısı zaten vardı — dairesel aksesuar §10'dan beri duruyordu.
—	(bulunan) iki widget	<code>JournalWidget</code> ve <code>RecoveryControlWidget</code> <code>WidgetBundle</code> 'a hiç eklenmemişti — Faz 22'den beri cihazda yoklardı.
BUILD-1	" <code>project.yml</code> yetkilidir"	Yanlıştı: dosyada saat hedefi yoktu, Faz 21'den beri yalnızca üretici betikte duruyordu. Eklendi.
C8	"HealthKit antrenmanlarını Hyrox istasyonlarına sınıflandır"	Veri bunu taşımaz. Kızak itme ile wall ball, ikisi de "fonksiyonel kuvvet antrenmanı, 42 dakika" olarak gelir. Sınıflandırma reddedildi , yerine tekrar tanıma yapıldı.

ÇIKARILAN DERS

Bir grep sayısı bir ölçüm değildir. A9'un imzalı aralıkları (`OSSignposter`) ve iş sayısını doğrulayan testler tam olarak bunun için var: bir sonraki denetim, sayfada iyi görünen bir tahmin yerine sayıya bakabilsin.

11.5 Sertleştirme turu — altı adım

v4 kapandıktan sonra altı başlıkta bir denetim turu daha yapıldı: eşzamanlılık, App Group, kilitli cihaz, saat dayanıklılığı, laboratuvar OCR'ı ve bilimsel sınırlar. Turun ürettiği şey yeni özellik değil, **on bir gerçek hata** — yedisi sessizce yanlış davranan, dördü hiç çalışmayan.

ADIM	BULGU	NEDEN GÖRÜNÜYORDU
1	<code>DayRecordCache</code> eşzamanlı isteklerde deliniyordu	Aktör her <code>await</code> 'te yürütücüyü bırakıyor; sıralı çağrıda 4→1, eşzamanlı çağrıda 4→4. Sayaç "1" diyordu çünkü çağırılar yarıştan önce sayılıyordu.
1	<code>runQuery</code> iptalde kalıcı askıda kalıyordu	İptal, sürekliliğin kaydından önce gelirse resume edecek kimse kalmıyordu. Hızla hata vermesi gereken okuma süreç boyunca asılı kalıyordu.
1	İki fan-out sınırsızdı	Yalnızca <code>MetricKind</code> bugün dört duruma sahip olduğu için 4'te kalıyorlardı.
2	Widget yenileme hattı hiç yoktu	Anlık görüntü yazılıyordu ama <code>WidgetKit</code> 'e haber verilmiyordu — widget önceki sabahın skorunu çiziyordu.
2	<code>project.yml</code> 'de widget hedefi <code>PendingJournalStore</code> 'suz	Yetkili dosyadan uzantı derlenmezdi; üreteç betiğinde vardı, ikisi karşılaştırılmıyordu.
3	Kilitli sabahta gün kaybediliyordu	<code>HealthKit</code> deposu kilitliken şifreli. Görev uyanıyor, hata alıyor, başarısızlık bildiriyor — ve sistem bunu öğrenip bir sonraki uyanmayı seyrekleştiriyor.
3	Aynı <code>HKEError</code> switch'inin üç kopyası	Yeni bir durumun ikisine eklenip üçüncüsünde unutulacağı şekil.
4	Uzun seansta kare karmaşıklık	Dört saatlik seansta ~1,24 milyar eleman ziyareti — saatin var olma sebebi olan aktivitenin ortasında, pil ve ısı olarak.
4	Telefon menzil dışındayken bitiş kayboluyordu	Gönderilemeyen yük loglanıp unutuluyordu; kare için doğru, sonuncusu için <code>Live Activity</code> 'nin koşudan saatler sonra çalışması demek.
5	Eğik taramada satırlar bölünüyordu	1,2° dönme beş satırı sekize çıkarıyor. Kopan değer kusursuz ayrıştırılıyor ve yalnızca yanlış sayı oluyor — aşağı akışta hiçbir şey bunu görmüyor.
5	Büyük harfli aksanlı ünlüler katlanmıyordu	"Â" tabloyu ıskalayıp "â" olarak kalıyordu; küçük harfli ikizi "a" veriyordu. Raporlar adları büyük harfle basar.
6	NORM-1 tutmuyor	Kontrol edilebilen dört hücrenin ikisi yanlış. Özellik doğrulanana kadar kapatıldı.
6	Minetti yorumu 8 puan sapıyordu	İki yorum dönüm noktasını "yaklaşık -%10" diyordu; eğri -%18,1'de dönüyor.

TURUN DESENİ

On üç bulgunun onu **çalışan koda bakarak** değil, *çalışmayan ama şikâyet etmeyen* koda bakarak çıktı. Aktör yeniden-girişi, kaydedilmemiş bir widget, kilitli bir cihaz, eğik bir tarama, doğrulanmamış bir tablo — hiçbirisi çökmez, hiçbirisi log basmaz, hepsi yanlış bir sayı üretir. Bu yüzden turun kalıcı ürünü düzeltmeler değil, **onları bir daha sessiz kalmaktan alıkoyan üç şey**: sayım testleri, [Scripts/check-target-sources.py](#), ve belgelenmiş her sabitin artık bir teste bağlı olması.

GELİŞTİRME SÜRECİ

12.1 Depo düzeni

```
zenithra/
├── Zenithium/                // ana uygulama – 239 dosya
│   ├── App/ Domain/ Engines/ Models/ Persistence/
│   ├── Health/ Orchestration/ Intelligence/ Labs/
│   ├── Reports/ Intents/ Connectivity/ Endurance/ Live/ Support/
│   ├── ViewModels/ Views/
│   └── Resources/Zenithium.xcassets/    // üretilen renk setleri
├── ZenithiumWatch/          // watchOS – 7 dosya
├── ZenithiumWidgets/        // widget uzantısı – 9 dosya
├── ZenithiumTests/          // 48 dosya, 66 paket, 588 test
├── Scripts/
│   ├── generate-project.py    // deterministik .pbxproj üretici, 4 hedef
│   └── generate-colors.py    // palet tablosu → catalog + Swift enum
├── docs/                    // 7 belge
└── project.yml              // XcodeGen tanımı
```

12.2 Üreteç betikleri

İki şey elle yazılmıyor, üretiliyor. **Xcode proje dosyası** — `generate-project.py` deterministik bir `.pbxproj` üretir; birleştirme çakışması çıkmaz ve dört hedefin dosya listesi elle senkronize edilmez. **Renk varlıkları** — `generate-colors.py` tek bir Python sözlüğünden hem 19 `.colorset` JSON'unu hem `ZenithiumColorAsset` enum'unu üretir; bir Swift adı olmayan renk setine işaret etmek imkânsızdır.

`project.yml` XcodeGen tanımını taşır ve üreteçle birlikte tutulur. ASSUMPTION BUILD-1 bu dosyanın yetkili olduğunu söylüyordu; v4'te bunun bir kez yanlış olduğu bulundu ve düzeltildi (bkz. §11.4).

12.3 Belgeler

BELGE	İÇERİK
ROADMAP.md	Faz 0–32 ürün yol haritası; her fazın kabul ölçütleri
ROADMAP-V4.md	Performans/tasarım/özellik dalgası, uygulama durumu ve düzeltmeler
PHASE0-ASSUMPTIONS.md	106 numaralı varsayım, gerekçeleriyle
PHASE0-ENGINE-CONSTANTS.md	Her motor sabitinin kaynağı ve gerekçesi
PHASE0-DEPENDENCY-GRAPH.md	Katmanlar arası bağımlılık haritası
PHASE0-MANIFEST.md • PHASE0-README.md	Dosya manifesti ve giriş belgesi
design-direction.html	Görsel yön referansı

12.4 Bilinen açıklar

AÇIK	DURUM
Hiç derlenmedi	En büyük açık. İlk <code>xcodebuild</code> çalıştırması gerekiyor; hatalar geri bildirildikçe düzeltilecek.
Hiçbir test koşturulmadı	588 test yazıldı, hiçbir çalıştırılmadı. Matematik Python'da ayrıca doğrulandı ama bu test koşumu değildir.
ASSUMPTION NORM-1	FRIEND VO ₂ max tablosu yayımlanmış kaynağa karşı doğrulanmalı.
App Group sağlama profili	Kimlik <code>group.com.projectcagla.zenithra</code> olarak dört yere yazıldı ve bir betikle korunuyor. Kalan iş hesap tarafında: grubun geliştirici hesabında oluşturulması ve üç hedefin sağlama profiline eklenmesi.
Paket kimliği ile grup kimliği farklı önek taşıyor	Paketler <code>com.zenithium.*</code> , grup <code>group.com.projectcagla.*</code> . Teknik olarak sorun değil — bir App Group herhangi bir önek taşıyabilir — ama App ID oluşturulmadan önce verilecek bir karar.
Uygulama simgesi yer tutucu	Gönderilebilir ama nihai değil; tasarımcı çizimiyle değiştirilmeli.
Cihazda görsel doğrulama yok	Hiçbir ekran gerçek bir cihazda veya simülatörde görülmedi; yerleşim ve tipografi kararları koddan okunabilir ama görülmedi.
Uzaktan itme (push) engelli	Bu oturum boyunca <code>git push</code> 403 döndü (GitHub App yazma izni yok). 45 commit yerelde duruyor; teslimat arşiv dosyasıyla yapıldı.

12.5 Sonraki adımlar

- Ön kontrol.** `./Scripts/preflight.sh` — dört kontrol ve proje üretimi, tek komut.
- Derle.** macOS'ta `xcodebuild`; çıkan hataları geri bildir. Kontroller isim çözümlemesine bakıyor, tip çıkarımına değil — hata çıkması hâlâ bekleniyor.
- Testleri koştur.** 588 testin ne kadarının geçtiği, kodun gerçek durumu hakkındaki ilk dürüst sinyaldir.
- Cihazda gör.** Özellikle kas haritası, sirkadiyen yay, ve aydınlık temanın gerçekten çalıştığı — `UIUserInterfaceStyle` kaldırıldı ama hiçbir ekran aydınlık zeminde görülmedi.
- App Group'u oluştur** ve üç hedefin sağlama profiline ekle. Paket kimliği önekinde de burada karar ver.
- NORM-1'i doğrula** — tabloyu yayımlanmış PDF'ten aktar ve `isPublicationVerified`'i çevir, ya da karşılaştırmayı kapalı bırak.
- Simgeyi değiştir** tasarımcı çizimiyle.
- Yazma izni** alındığında dalı uzağa it.

12.6 Yayın öncesi durum

Yayın kararından önce depo baştan sona tarandı. İki gerçek derleme hatası ve dört yayın engeli çıktı; hepsi kapatıldı.

BULGU	TÜR	NEDEN GÖRÜNÜYORDU
<code>ReportView</code> UIKit import etmiyordu	DERLEME	<code>UIActivityViewController</code> ve bir <code>UIViewControllerRepresentable</code> kullanıyor ama yalnızca SwiftUI import ediyordu. Dosya, modülde başka bir şeyin UIKit'i içeri çekmesine bağlı derleniyordu — bağımlılık değil, tesadüf.
İki <code>ShareLinkSheet</code>	OKUNABİLİRLİK	İkisi de <code>private</code> olduğu için yasal; ama biri UIKit köprüsü, öbürü saf SwiftUI. Arşiv olanı <code>ArchiveShareSheet</code> oldu.
Uygulama simgesi yoktu	YAYIN	Simgesiz uygulama App Store'a gönderilemez. Varlık kataloğunda yalnızca renkler vardı.
Gizlilik bildirimi yoktu	YAYIN	<code>PrivacyInfo.xcprivacy</code> Mayıs 2024'ten beri zorunlu ve her paketin kendisinininki gerekiyor — uzantı uygulamanınkiyle kapsanmıyor.
<code>UIUserInterfaceStyle = Dark</code>	YAYIN	B6'nın eklediği aydınlık palet ve üç seçenekli görünüm ayarı tek bir Info.plist anahtarıyla geçersiz kalıyordu.
<code>armv7</code> ve boş açılış rengi	YAPILANDIRMA	32-bit ARM yeteneği bir iOS 18 uygulamasında yanlış; açılış ekranı boş bir renk adına işaret ediyordu.
Geliştirme dili <code>en</code>	YAPILANDIRMA	Arayüzün tamamı Türkçe.

UYGULAMA SİMGESİ

Uygulamanın kendi görsel kimliğinden üretildi: koyu zemin, spektral toparlanma yayı ve ortada zirve işareti — yani her ekranda görünen şeklin kendisi. iOS 18'in üç varyantı da mevcut (açık, koyu, tinted). **Bir tasarımcı çizimiyle değiştirilmeli;** bu, gönderilebilir olmayı sağlayan bir yer tutucudur, nihai kimlik değil.

GİZLİLİK BİLDİRİMİ

Üç paketin üçünde de var ve **içeriğinin tamamı boş** — boşluğun kendisi ürün. Ağ katmanı, hesap ve analitik olmadığı için toplanacak veri yok, izlenecek alan adı yok. Tek beyan `UserDefaults` / `CA92.1`: App Group süiti.

İDDİA DENETLENİYOR

Bir gizlilik bildirimi, tam olarak bir yıl doğru kalıp birinin çökme raporlayıcısı eklediği gün sessizce yanlışlaşan türden bir dosyadır. `Scripts/check-privacy-manifest.py` iddiayı kaynağa karşı sınıyor: 246 dosyada ağ API'si yok, SPM bildirimi yok, ve gerekçe gerektiren API'lerde kullanım ile beyan birebir örtüşüyor. *Fazla* beyan da eksik beyan kadar yanlış sayılıyor, çünkü olmayan bir uygulamayı tarif ediyor.

12.7 Derleme öncesi kontroller

Bu konteynerde Swift derleyicisi yok, dolayısıyla "derlenmeye hazır" doğrulanamaz. Doğrulanabilen her şey `Scripts/preflight.sh` altında toplandı.

KONTROL	YAKALADIĞI
<code>check-target-sources.py</code>	<code>project.yml</code> ile üretcin bir hedefin neyi derlediği konusunda ayrışması; App Group kimliğinin Swift ile üç yetkilendirme dosyası arasında kayması
<code>check-symbols.py</code>	Bir hedefin derlemediği bir tipi kullanması; çift üst düzey tanım; tipte olmayan argüman etiketi; yasaklı yapı; parantez dengesizliği
<code>check-privacy-manifest.py</code>	Bildirimin kaynaktan az şey iddia etmesi
<code>generate-project.py</code>	Depodaki <code>.xcodproj</code> 'un kaynak ağacına göre bayatlaması

GÜRÜLTÜLÜ BİR KAPI, KAPI DEĞİLDİR

`check-symbols.py` 'nin ilk sürümü **272** bulgu veriyordu ve neredeyse hepsi yanlış pozitif: iç içe parantezleri düzenli ifadeyle ayırtırmak `Foo(bar: Baz(qux: 1))` içindeki `qux` 'u `Foo` 'ya yazıyordu. Gerçek parantez eşlemeyle `272` → `3`, makro üretimi `init` 'ler tanınınca `3` → `0`.

Bu düzeltme isteğe bağlı değildi. İnsanların görmezden gelmeyi öğrendiği bir kontrol, hiç olmayan bir kontrolden kötüdür — çünkü kapsandığı yanılışmasını taşır.

BUGÜNKÜ ÇIKTI

```

OK    App Group 'group.com.projectcagla.zenithra': Swift ve üç yetkilendirme dosyası aynı
OK    ZenithiumWidgets: 13 paylaşılan kaynak, iki dosya da aynı
OK    ZenithiumWatch: 12 paylaşılan kaynak, iki dosya da aynı

Zenithium      231 dosya, 513 tip · her tip referansı hedefin içinde çözülüyor
ZenithiumWidgets 88 dosya, 247 tip · her tip referansı hedefin içinde çözülüyor
ZenithiumWatch  85 dosya, 228 tip · her tip referansı hedefin içinde çözülüyor
ZenithiumTests  277 dosya, 587 tip · her tip referansı hedefin içinde çözülüyor

OK    her adlandırılmış argüman tipinde mevcut
OK    yasaklı yapı ve denge temiz
OK    246 dosyada ağ API'si yok · SPM bildirimi yok · 3/3 pakette gizlilik bildirimi

Wrote Zenithium.xcodeproj/project.pbxproj - 230 app, 88 extension, 85 watch, 46 test

```

YİNE DE DERLEME DEĞİL

Bu kontroller isim çözümlemesi ve yapılandırma tutarlılığı bakıyor.

Tip çıkarımını, aşırı yükleme seçimini, jenerik kısıtları ve Swift 6 izolasyon kurallarını denetlemiyorlar — onlar için tip denetleyicisi gerekiyor ve bu konteynerde yok. İlk `xcodebuild` çalıştırmasında hata çıkması hâlâ normaldir; bu turun yaptığı, o listeyi kısaltmak.

FAZ LİSTESİ

Uygulama 33 fazda inşa edildi. Faz 0–9 çekirdeği kurdu (şema, taban çizgisi, toparlanma, uyku, zorlanma, yorgunluk, sirkadiyen, arayüz iskeleti); Faz 10–32 aşağıdadır.

FAZ	NE İNŞA EDİLDİ
10	Günlük ve korelasyon motoru
11	Sağlık izleme katmanı — 18 hayati işaret
12	Regl döngüsü farkındalığı, faza duyarlı taban çizgileri
13	Stres ve gün içi yük
14	Antrenman zekâsı — ACWR, monotonluk, form (tüm mercekler ortak)
15	Dayanıklılık merceği — kritik hız, tempo bölgeleri, yarış tahmini
16	Hibrit mercek — Hyrox/CrossFit, kompanse koşu cezası
17	Kuvvet merceği — tonaj, 1RM, dengesizlik
18	Sağlık merceği
19	Reçete motoru — beş adımlı karar zinciri
20	Planlama ve periyodizasyon
21	Apple Watch uygulaması
22	Sistem entegrasyonu — widget, App Intents, arka plan
23	Laboratuvar zekâsı — PDF/OCR, 51 biyobelirteç
24	Cihaz içi yapay zekâ — anlatıcı + güvenlik filtresi
25	Görsel sistem profesyonelleşmesi
26	Sağlık belge kasası
27	Hekime rapor (PDF dışı aktarım)
28	Erken uyarı ve anomali tespiti
29	Hareket kalitesi ve uzun ömür
30	Çevresel bağlam — gün ışığı, saat dilimi
31	Sistem derinliği
32	Sakatlık ve ağrı haritası
v4	Performans (9 madde) · tasarım (10 madde) · özellikler (11 madde)

VARSAYIM KAYIT DEFTERİ

Belgelenmiş bir cevabı olmayan her karar numaralandırıldı. Numara hem `docs/PHASE0-ASSUMPTIONS.md` 'de hem ilgili kaynak dosyada geçer, böylece bir kararın gerekçesi kodun yanında durur.

ÖRNEK KAYITLAR

NUMARA	İÇERİK
BASE-3	Zenithium bilek sıcaklığının <i>mutlak</i> okumasını tabanlar ve ΔT 'yi kendisi türetir, çünkü <code>appleSleepingWristTemperature</code> bazı kaynaklarda mutlak, bazılarında farktır.
CIRC-1	Sirkadiyen çapa uyku orta noktasına göre yerleştirilir; orta nokta hesaplanamıyorsa uyanma saati kullanılır.
CORR-1	Güven aralığı normal yaklaşımı ($z = 1,96$) kullanır. Telafi olarak minimum örneklem 5 yerine 6 tutulur ve gözlem sayısı her zaman gösterilir.
AI-1	<code>FoundationModelNarrator</code> yalnızca <code>FoundationModels</code> 'in var olduğu yerde derlenir; yokluğunda deterministik anlatıcı tek yol olur.
NORM-1	Riskli. FRIEND $VO_2\max$ tablosu yayımlanmış kaynağa karşı doğrulanmadı.
BUILD-1	<code>project.yml</code> hedef tanımlarında yetkilidir. <i>Bu iddia v4'te bir kez yanlış çıktı</i> — dosyada saat hedefi yoktu; eklendi.

Zenithium teknik dökümanı · Sürüm 0.1 · 21 Ağustos 2026 · İlk yayın
· `projectcagla/zenithra` · HEAD 272da6a · Bu belgedeki her ölçüm
depoda çalıştırılan bir komutun çıktısıdır.